

The Hermes Agent Masterclass

All twelve parts of Tony Simons' series, combined, indexed, drilled and annotated into one operator's reference

Why this exists: So the whole series can be learned end to end from one offline document instead of twelve separate articles, with every mechanism drilled to the level where a reader can operate it, debug it, and teach it.



SECTION

Orientation

This document is the whole of Tony Simons' twelve-part **Hermes Agent Master Class**, published on X between July 5 and July 20, 2026, combined into one offline reference and drilled deeper at every part.

Nothing from the original is dropped. Every mechanism the series names is here, every figure the author drew is here, and on top of that each part carries three additions that the serialized format could not give you:

- **A mechanism diagram.** The series describes the agent loop, the cron tick, the delegation tree and the profile boundary in prose. Here each one is also a real graph you can read in five seconds.
- **Reference tables.** Numbers scattered across a paragraph (iteration budgets, compression thresholds, curator timers, token costs) are pulled into tables you can look up later without rereading the prose.
- **An operator drill.** One concrete exercise per part. Reading about the agent loop teaches you nothing you can use on Tuesday. Running a command that proves the loop behaved the way the article said it would is what makes it yours.

WHOSE WORDS ARE WHOSE

The substance and the structure are Tony Simons' work, sourced from the articles linked in Appendix I. The original article artwork is not reproduced; every illustration and diagram here was produced for this compilation, as were the tables, the operator drills, the cross-references and the three configuration appendices. Where an addition makes a claim the series did not, it is marked as an addition rather than blended into the source material.

How to read this

There are three honest ways through this document and they suit different weeks.

PATH	WHAT YOU READ	TIME	WHO IT IS FOR
Foundation	Parts 1, 2, 3, 5, and Appendix D	About 40 minutes	Anyone who wants the agent working correctly before adding anything
Operator	Foundation plus Parts 4, 6, 7, 12 and Appendix F	About 90 minutes	Someone running Hermes daily who wants it running without them
Full stack	All twelve parts plus every appendix	Half a day	Someone building multi-agent infrastructure on Hermes

PATH	WHAT YOU READ	TIME	WHO IT IS FOR
Configure it	Appendices F, G and H on their own	About 25 minutes	Someone who wants the identity files, a first skill, and a prompt that produces a working system

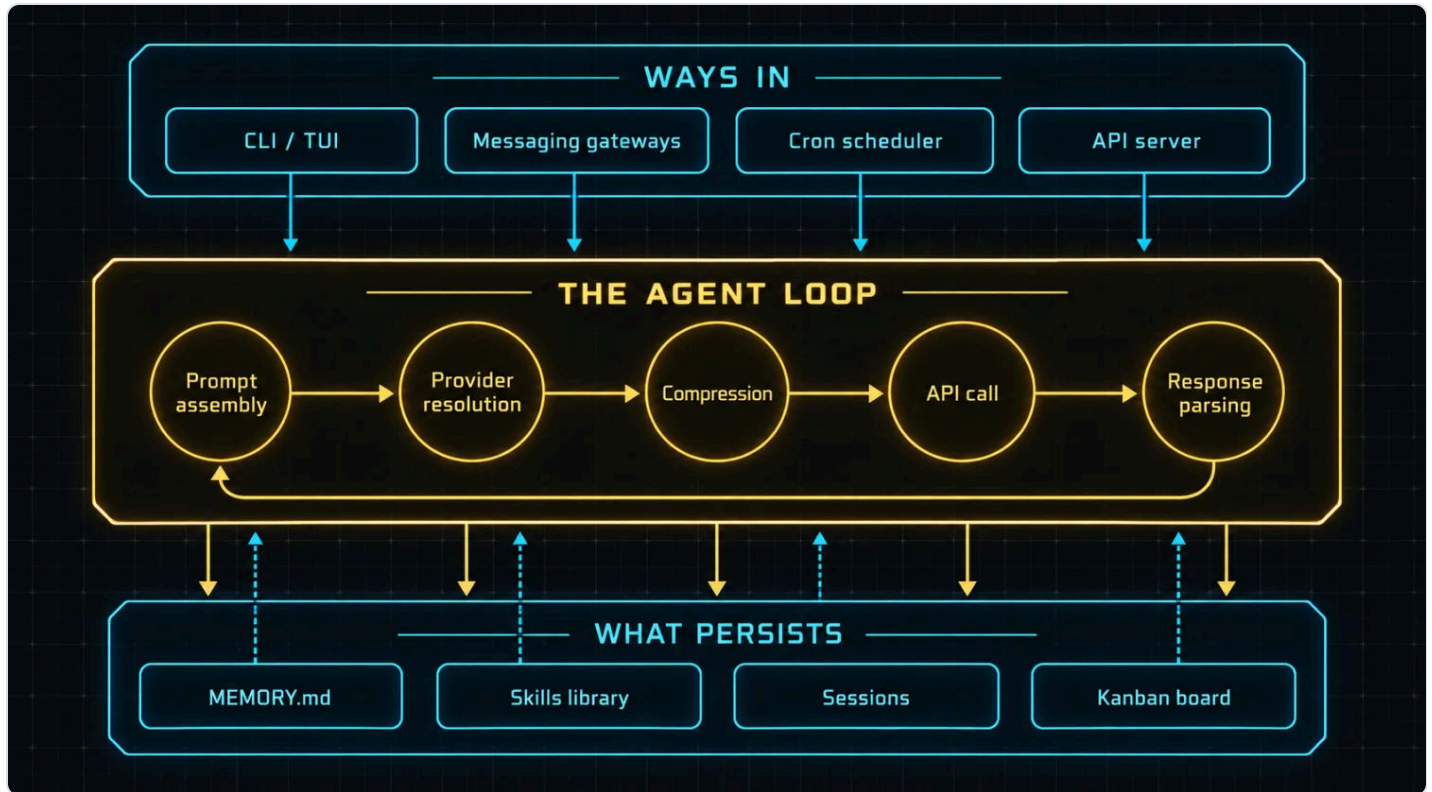
The parts genuinely build. Part 3 assumes Part 2's session persistence actually works. Part 6 assumes Part 4's skills exist. Part 10 assumes Part 11's profiles. Reading out of order is possible but each part will quietly reference a foundation you have not laid.

The one sentence version

A chatbot predicts the next token. Hermes runs a process: it assembles a layered prompt, resolves a provider, calls the model, dispatches whatever tools the model asked for, feeds the results back, and repeats until the model produces text instead of another tool call. Then it saves the session, updates its memory, and is ready to resume later. Everything else in these twelve parts is a consequence of that loop existing.

The System in One Picture

Twelve articles describe nine subsystems. They fit together like this.



Read it as three rings. The **middle ring is the loop**, and it is the only part that is always running. The **top ring is every way a turn can start**, and the loop cannot tell them apart. The **bottom ring is what survives the turn**, which is the entire reason the system compounds instead of merely responding.

The dotted lines matter as much as the solid ones. Memory, skills, sessions and the board feed *into* prompt assembly, and the background review feeds back into memory and skills. That cycle is the learning system, and it is the difference between an agent and a chatbot with a plugin folder.

The Twelve Parts, Indexed

#	PART	THE ONE THING IT TEACHES	DRILL IN THIS DOC
1	How Hermes Actually Processes Work	The turn is a five-stage loop, not a single inference	Trace one turn end to end
2	The Choices That Compound	Deployment home, session persistence and tool reach are the only day-one decisions that matter	The three-tool smoke test
3	The Learning System	Memory holds facts, skills hold procedures, the background review writes both	Force a memory write and verify it survives
4	Skills as Executable SOPs	A skill is a markdown file with a trigger, a procedure, pitfalls and a verification	Write one real skill
5	Tools and Toolsets	Capability is dynamic and gated by check_fn, not fixed at install	Audit your live tool surface
6	Cron as Infrastructure	A scheduled job runs in a fresh session, so the prompt must be self-contained	Ship a zero-token watchdog
7	Messaging Gateways	One gateway process, 20+ platforms, portable sessions	Cross-surface a single session
8	Delegation and Subagents	Children get isolated context and their own budget; only the summary returns	Run a three-way parallel research fan-out
9	Browser and Computer Use	The agent acts on interfaces that were never built for APIs	Hybrid-route a local and a public page
10	Kanban as a Coordination Model	A durable board is how multiple profiles coordinate without talking	Stand up a three-profile pipeline
11	The Admin Layer	A profile is a whole independent agent, not a config preset	Clone a profile and prove isolation

#	PART	THE ONE THING IT TEACHES	DRILL IN THIS DOC
12	What Breaks and What to Skip	Context is the first wall; integrations fail silently	Build the weekly tool-surface canary

Vocabulary, front-loaded

The series uses these terms from Part 1 onward. Knowing them before you start saves rereading.

TERM	WHAT IT ACTUALLY MEANS
Agent loop	The five-stage cycle run per turn: assemble, resolve, compress, call, parse. Repeats while the model returns tool calls
Turn	One pass through the loop. A tool call costs a turn, so one user message can cost many
Iteration budget	The cap on turns in a single task. 90 for the main agent, 50 for a subagent
Toolset	A named bundle of tools granted to a surface. The CLI gets a broad one, a phone bot a narrow one
check_fn	A per-tool availability test run at schema build time. Fails, and the model never sees the tool exists
Progressive disclosure	Loading a skill index at session start and the skill body only on demand
Profile	A fully independent agent: its own config, memory, skills, sessions, cron, gateway and credentials
Stable / context / volatile	The three ordered prompt tiers. Stable is cacheable identity, volatile is memory and timestamp
Session lineage	The chain linking a compressed session to the one it came from
Background review	A forked agent that inspects each finished turn and proposes memory and skill writes
Curator	The scheduled garbage collector that ages, archives and consolidates skills

TERM

WHAT IT ACTUALLY MEANS

No-agent mode	A cron job that runs a script and delivers stdout, with no LLM call at all
----------------------	--

How Hermes Actually Processes Work



Every AI chatbot you have used has the same architecture. Type a message, the model generates a response. One step, done.

Hermes is different in a way that is not obvious from screenshots. When you send it a message it runs a structured process, prompt assembly through provider resolution, API call, tool dispatch, result evaluation and context persistence, before it responds. Some messages trigger ten tool calls, each feeding back into the model for another round of reasoning. Then the session is saved, memory is flushed, and the whole thing is ready to resume later.

That is an **agent loop**, and understanding it is the key to understanding why this tool compounds in ways a chatbot does not.

The loop versus the chat

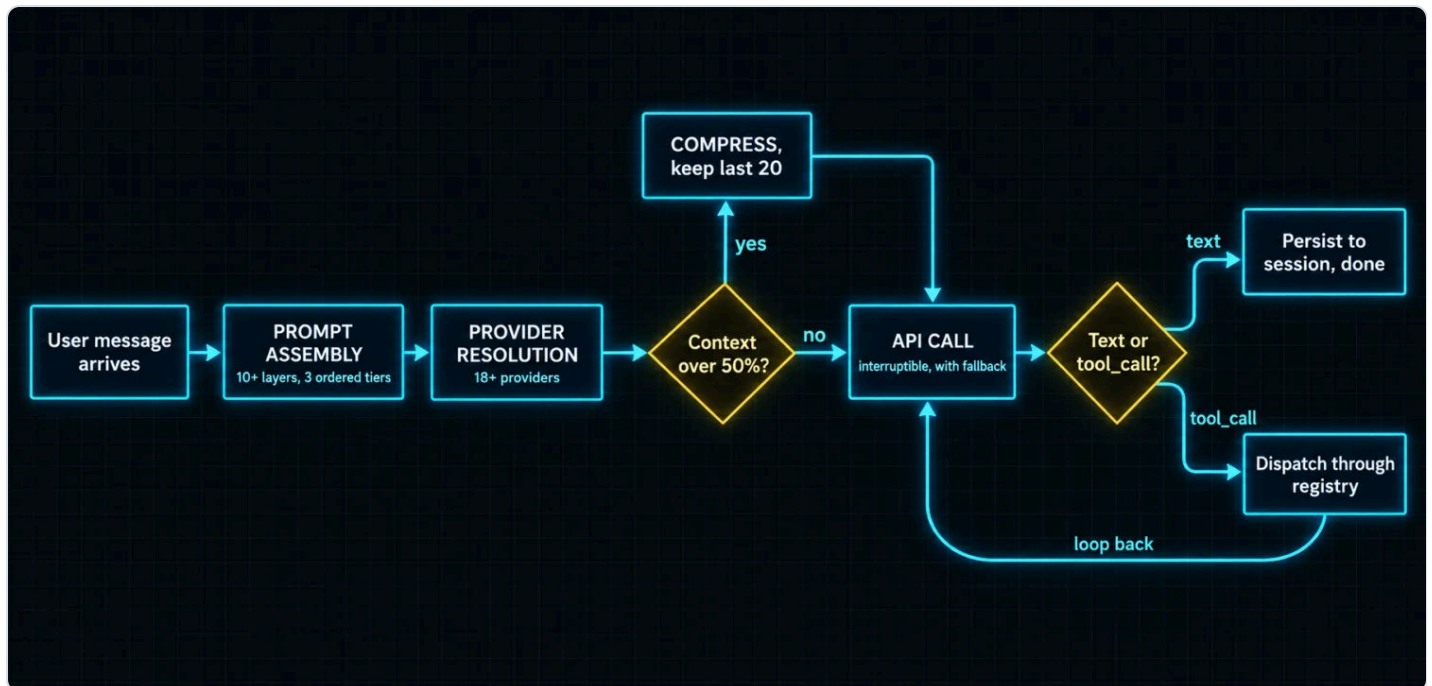
A chatbot's architecture is one arrow: user message, model predicts, response. Every input is a fresh inference against static training data. The model does not run anything and does not check anything. It guesses, based on what it learned during training.

Hermes has a loop. The `AIAgent` class lives in `run_agent.py` and handles the entire lifecycle of a single turn: prompt assembly, provider selection, API call, tool dispatch, compression, fallback and persistence. It supports three API execution modes, OpenAI chat

completions, OpenAI Codex/Responses, and native Anthropic Messages, and converges all of them into one internal message format.

That last detail is worth pausing on. **Three wire protocols, one internal representation.** It is why switching providers does not change how skills, memory or tools behave, and why the rest of the system can be written without knowing which vendor is answering.

The five stages, in order, every time



1. Prompt assembly. The system builds your context from ten-plus layers. SOUL.md for identity. Skills for procedural knowledge. Memory and user profile snapshots. Context files from your project directory. Platform hints for where you are chatting from. All assembled into three ordered tiers: stable, context, volatile.

2. Provider resolution. Maps your provider and model selection to the right API endpoint, API key and mode. Handles 18+ providers, OAuth flows and credential pools.

3. Preflight compression. If the conversation exceeds 50% of the model's context window, Hermes compresses *before* making the API call. Middle turns are summarized, the last 20 messages are preserved intact, and a new session lineage ID is generated.

4. API call. The assembled context goes to the model. The HTTP request runs in a background thread with an interrupt event watching it. You can cancel mid-flight with a signal, a `/stop` command, or by sending a new message. On a 429 or 5xx, Hermes checks its fallback provider list and tries the next one.

5. Response parsing. Text means that is the answer and it gets persisted. Tool calls mean the loop continues.

WHY COMPRESSION IS PREFLIGHT, NOT REACTIVE

Compressing before the call rather than after a failure means you never eat a context-length error. It also means the compression decision is made with full knowledge of what is about to be sent. The cost is that compression is lossy and silent: a summarized middle is not the original, and nothing tells you it happened except a new lineage ID. Part 12 returns to this as the first wall you will hit.

Where tools enter

This is the part that is fundamentally different from a chatbot. When a language model determines it needs to *do* something, run a command, search the web, write a file, read a document, it returns a `tool_call` instead of text. Hermes catches that and dispatches it through a central registry at `tools/registry.py`.

The registry holds **70+ registered tools across roughly 28 toolsets**. Each tool file calls `registry.register()` at import time with its name, schema, handler function, availability check and metadata.

When a `tool_call` arrives, three things can happen:

PATH	WHICH TOOLS	WHY IT IS SEPARATE
Intercepted by the agent loop	memory, todo, session search, delegation	They need direct access to agent state, not just arguments
registry.dispatch()	Everything else	Looks up the handler, checks availability via check_fn, executes, returns JSON

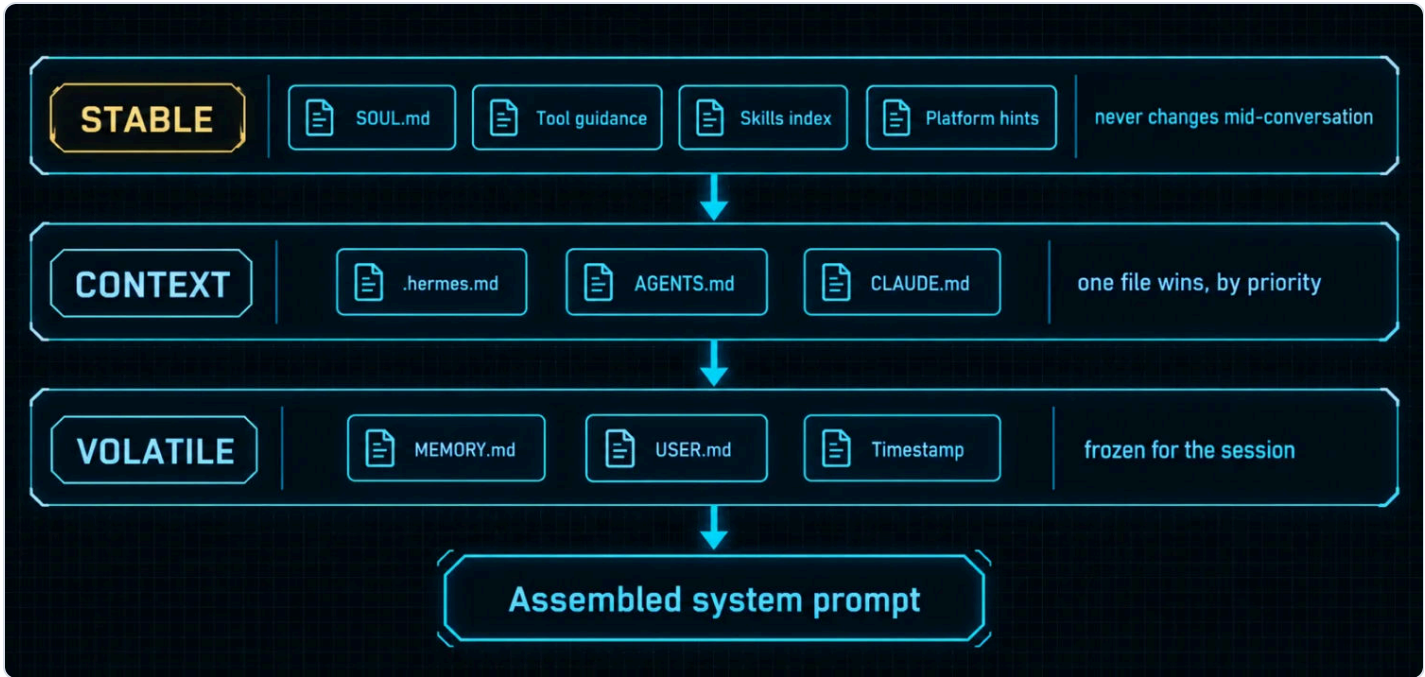
PATH	WHICH TOOLS	WHY IT IS SEPARATE
Error wrapping	Both paths	dispatch() catches handler exceptions, handle_function_call() catches dispatch exceptions

That double error wrap has a specific consequence: **the model always receives a well-formed result, never an unhandled error**. A tool that throws becomes a result describing the throw. The model can then reason about the failure instead of the loop dying.

Multiple tool calls from a single model response run **concurrently** via a thread pool executor. The exception is tools marked `interactive`, like `clarify`, which force sequential execution. Once all results come back they are appended as tool-role messages and the loop returns to the API call with the new context. This continues until the model returns text or hits the iteration budget.

The prompt architecture is the product

The reason Hermes gets better over time is not magic, it is structural. The system prompt is built as three ordered tiers.



TIER	CONTENTS	CHANGES WHEN
Stable	SOUL.md identity, tool guidance, skills index, environment and platform hints	Never mid-conversation
Context	Project .hermes.md or AGENTS.md or CLAUDE.md, plus any caller system message	Per working directory, one type only, discovered by priority
Volatile	Memory snapshot, user profile snapshot, external memory block, timestamp and session line	Between sessions, frozen during one

Memory is written to disk mid-session but does not mutate the cached system prompt until a rebuild path runs: new session, compression, or explicit invalidation. This keeps the prompt prefix stable for provider-side caching.

The separation is deliberate and buys three things:

1. The first part of the prompt, identity and tools and skills, benefits from API-level prompt caching.
2. Memory changes do not break that cache mid-conversation.
3. Skills, context files and platform hints each have a defined slot with defined precedence.

CONTEXT FILE PRECEDENCE IS A REAL GOTCHA

Only ONE project context type loads, discovered by priority from the working directory. A `.hermes.md` in your project root beats an `AGENTS.md` in the same directory. A `CLAUDE.md` loads only if neither of the other two exists. If you have all three and wonder why your CLAUDE.md is being ignored, this is why.

Five design choices that explain the behavior

PRINCIPLE	WHAT IT MEANS IN PRACTICE
Prompt stability	The system prompt does not change mid-conversation. No cache-breaking mutations unless you switch models with <code>/model</code>
Observable execution	Every tool call is visible: a spinner in the CLI, progress messages in Telegram, callbacks in Discord

PRINCIPLE	WHAT IT MEANS IN PRACTICE
Interruptible	API calls and tool execution cancel mid-flight cleanly. Send a new message and the old request is abandoned, not force-quit
Platform-agnostic core	One AI-Agent class serves CLI, gateways, the ACP editor integration, batch, and the API server. Platform differences live in the entry point
Loose coupling	MCP servers, plugins, memory providers and RL environments all use registry patterns and check_fn gating. A failed plugin does not take the agent down

What this changes about how you use it

Four consequences follow directly from the architecture, and each one changes a habit.

Skills load into the stable tier, so they are always available but do not change mid-conversation. If you want the agent to adopt new expertise, add or switch skills *between* sessions, not during one.

Context files load by priority from your working directory. Structure your project context accordingly rather than scattering three files and hoping.

Tool calls are concurrent by default. If you ask for four independent things, they happen simultaneously. Write prompts that exploit that: "check the disk, the last deploy, the open PRs and the error log" is one turn, not four.

The iteration budget is real. Default 90 turns, each tool call counting as one. A complex task needing 15 tool calls eats 15 of your 90. Subagents get independent budgets capped at 50. For long workflows, budget the tool calls, not the conversation turns.

BUDGET	DEFAULT	COUNTED AGAINST
Main agent iterations	90 turns	One task in one session
Subagent iterations	50 turns	Each child independently
Preflight compression trigger	50% of context window	Before the API call
Gateway auto-compression	85% of context window	During a gateway session

BUDGET	DEFAULT	COUNTED AGAINST
Messages preserved on compression	Last 20	Everything older is summarized

OPERATOR DRILL · TRACE ONE TURN

Ask Hermes something that forces exactly three tool calls, for example: "Check my disk usage, then search the web for the current version of Bun, then write both answers to /tmp/hermes-drill.txt." Watch the CLI. You should see three tool executions, the first two running concurrently, then the file write. Afterwards, confirm the file exists and holds both answers. You have now watched stages 4 and 5 loop three times against one message, which is the entire lesson of Part 1 made concrete.

The compounding claim, stated precisely

Skills compound because they are loaded as stable context the model never forgets it has. Memory compounds because it is written to disk mid-session but snapshotted at session boundaries. The tool system grows because new tools self-register at import time without manual wiring.

Chatbots predict the next token. Hermes runs a process. That is the difference that compounds, and everything in the remaining eleven parts is downstream of it.

The Choices That Compound



Most people spend their first hour with Hermes picking a provider and running the installer. Both things matter. Neither is the decision that will matter a month from now.

The choices that compound are three: **where does your agent live, does it persist sessions across restarts, and can it actually use its tools**. Get these right on day one and everything after gets easier. Get them wrong and you will spend the next month wondering why the learning loop never seems to kick in.

Where your agent lives is the infrastructure decision

Hermes runs on six terminal backends and each one changes how you interact with the agent.

BACKEND	WHAT IT GIVES YOU	THE TRADEOFF
Local desktop	Native app on macOS and Windows, manages its own config, sessions and Python env. Sessions in SQLite, memory on disk, always available when the machine is on	Your agent dies with your laptop. Close the lid and Hermes goes quiet
Docker	Survives reboots and laptop swaps. Maps ~/.hermes to /opt/data. Runs as a supervised gateway service. The agent lives on your server	You must remember the volume mount. Forget it and the agent starts fresh every time

BACKEND	WHAT IT GIVES YOU	THE TRADEOFF
Daytona	Serverless, hibernates when idle, costs nearly nothing	Not reachable during hibernation
Modal	Serverless, same hibernation model. Good for batch and research	Same unreachability
SSH	The right call when you have a beefy remote machine and want commands running there	Keeps the agent away from its own code, which is a feature and a constraint
Singularity	HPC and scientific computing environments	Narrow by design

The pattern that works for most people is a progression, not a choice: **start local to learn the system, add Docker or a cloud backend when you want the agent running independently, then wire up gateways so you can reach it from anywhere.**

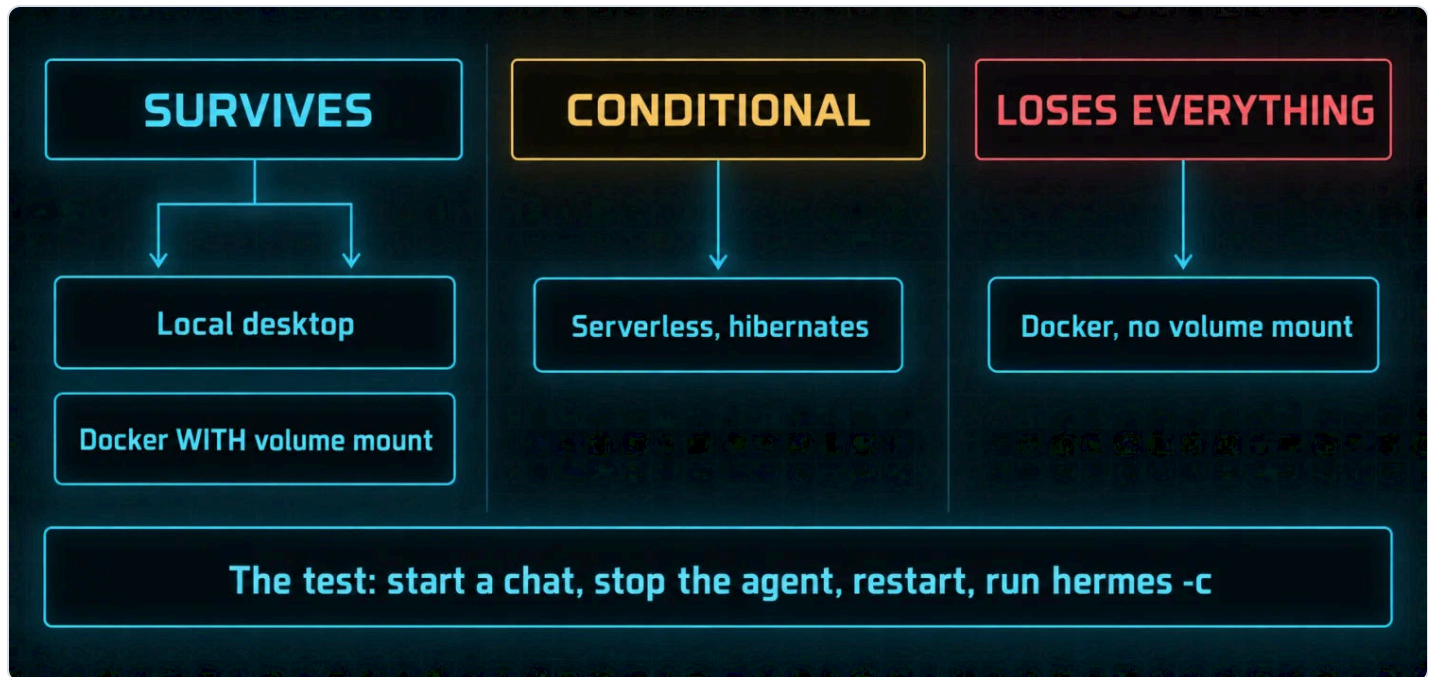
THE SINGLE VOLUME THAT HOLDS EVERYTHING

In Docker, `~/.hermes` mapped to `/opt/data` is the whole of your agent: config, sessions, skills, memories and gateway credentials. That is a genuinely good design, because it means you can pull a new image, destroy the container, and spin up a fresh one against the same data directory without losing anything. It also means one forgotten `-v` flag silently discards your agent's entire accumulated life.

Session persistence is the feature everyone discovers the hard way

Every conversation with Hermes is saved as a session, stored in a SQLite database at `~/.hermes/state.db`. Sessions get titles. They get full-text search via FTS5. They track lineage across compression events. The agent can search them, resume them, and hand them off between platforms.

This is the infrastructure that makes the learning loop work. Without it, every conversation starts from zero and the agent has no way to build on previous work.



On serverless backends the environment hibernates when idle and the agent's state hibernates with it. Sessions resume when it wakes, but the agent is not reachable during hibernation. That is fine for scheduled cron work. It is not fine if you want to ping the agent from Telegram and get an answer in real time.

The lesson, stated as the author states it: **session persistence is invisible when it works and catastrophic when it does not.**

The first real task should test tools, not chat

Here is the mistake almost everyone makes on day one. They ask Hermes a conversational question. "Write a poem about AI." The agent responds, the conversation looks good, and they assume everything works.

That test proves the model works. It does not prove the agent works. The thing that makes Hermes different from ChatGPT is the tool surface. If the tools are not wired up, you have paid for a chatbot with extra steps.

- ☐ Terminal: "What's my disk usage?" or "Show me the last 5 files modified in my project."
Real output means the terminal tool is working
- ☐ Web: "Find the latest news about [something current] and summarize it in three bullets."
Live results mean the web tool is working

- ☐ File: "Write today's date to a file called test.txt and tell me the absolute path." A file on disk means the file tools are working

If all three pass, the agent is functional and you can move on to memory, skills, cron and the rest. If one fails, fix it before doing anything else. A missing API key or a misconfigured backend will frustrate you for weeks if you do not catch it early.

DESIGN THE SMOKE TEST TO PROVE THE SURFACE, NOT EXHAUST THE BUDGET

Each tool call costs one of your 90 iterations. A smoke test that takes ten tool calls has spent an ninth of the budget proving something three calls would have proven. Keep it to three.

What actually compounds

The three choices are not the exciting part of setting up Hermes. They are the important part.

A local agent with persistent sessions and working tools is a **platform**. Every skill you add, every cron job you schedule, every memory the agent accumulates compounds against a stable foundation.

An agent that loses sessions on restart, cannot reach the web, or lives on a laptop that gets closed every night will never develop the compounding effect. It will always feel like a chatbot, because without these three things holding steady, that is all it is.

OPERATOR DRILL · PROVE PERSISTENCE, DO NOT ASSUME IT

Run the three-tool smoke test. Then start a distinctive conversation ("remember that my drill codeword is BASALT"), stop the agent completely, restart it, and run `hermes -c`. If the codeword comes back, your foundation is real. If it does not, stop reading this document and fix storage, because Parts 3 through 12 all assume this works.

The Learning System



Parts 1 and 2 covered the engine and the chassis. The engine fires, the chassis is stable. Now the question that actually separates Hermes from the field: **does it get better over time?**

That is not rhetorical. Most AI tools ship at a fixed capability level and stay there. ChatGPT today knows roughly what it knew last week. Claude Code is the same Claude Code it was on install day. The vendor might ship an update, but the tool itself does not learn from your work.

Hermes does. Not because the model trains on your data, but because of three systems that accumulate, formalize and maintain what the agent learns about you, your projects and your workflows.

Memory is the raw material

Memory in Hermes is not a log of everything that happened. It is a small, curated set of facts the agent keeps in context at all times, held in two files.

THE

FIELD	TYPE	NOTES
MEMORY.md	agent notes	environment facts, project conventions, tool quirks, completed work

FIELD	TYPE	NOTES
USER.md	your profile	preferences, communication style, technical level, pet peeves
combined cap	~1,300 tokens	tight by design, because memory costs tokens in every single prompt
MEMORY.md hard limit	2,200 characters	exceeding it returns an error, not a silent drop

The agent writes to memory automatically when it learns something durable. You correct it about a convention, it saves that. It discovers your project uses Go 1.22 and sqlc, it saves that. You ask it to remember your API key rotation schedule, it saves that.

Memory is a frozen snapshot at session start. Everything remembered is loaded into the system prompt as a block of text, available from the first message. Mid-session memory changes are persisted to disk but do not appear in the prompt until the next session. This is what keeps the prompt prefix stable for provider-side caching, exactly as Part 1 described.

When memory fills, and at 2,200 characters it will, the agent does not silently drop entries. The tool returns an error carrying the current entries and the usage count. The agent then consolidates: merging related entries, removing stale facts, making room. The error message shows exactly what is in memory so the agent can decide what to keep.

THE ECONOMICS THAT DRIVE THE DESIGN

Memory costs tokens in every prompt. Session search costs nothing until you run a query. So the agent uses session search for recall ("what did I learn about project X last week?") and saves to memory only what should *always* be in context. If you find memory filling with things you rarely need, that is the split being applied wrongly.

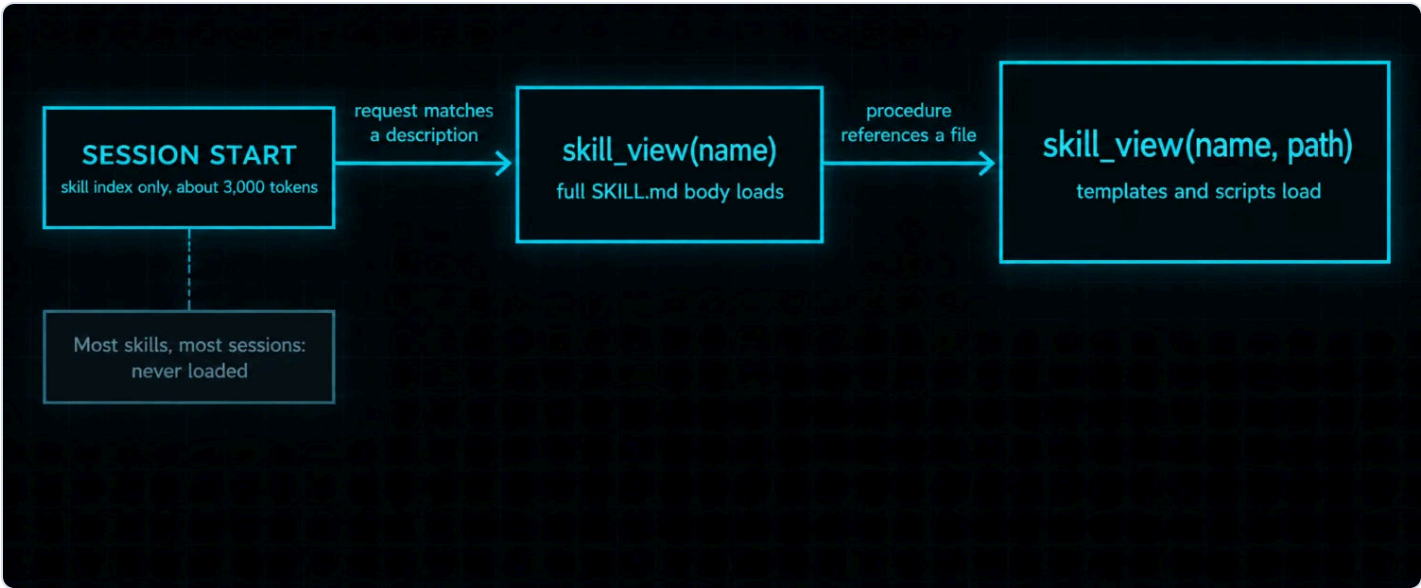
Skills are the procedures

Memory stores facts. Skills store procedures.

When the agent solves a novel problem in a multi-step workflow, five or more tool calls, significant back-and-forth, a correction from you, it can save the approach as a skill: a

markdown file in `~/hermes/skills/` with a name, description, step-by-step procedure, pitfalls section and verification steps.

Skills use **progressive disclosure** to minimize token overhead, and this is the mechanism that lets the library scale.



Multiple skills can be stacked in a single command. Running `/github-pr-workflow /test-driven-development fix issue #123` loads both skills and the agent follows both sets of instructions for the same task. For workflows you repeat constantly, skill bundles group several skills under a single slash command.

SOURCE	WHAT IT IS	HOW YOU GET IT
Bundled	Ships with Hermes, covers code review, PR management, research	Already there
Hub	Community-contributed	hermes skills install, after a security scan
Agent-created	The procedural memory of your specific work patterns	The agent writes them after complex tasks

The background review is invisible infrastructure

Memory and skills do not just accumulate passively. **After every conversation turn, Hermes forks a background review that examines what happened.**

The fork runs as a separate AI agent in its own prompt cache and never touches the active conversation. It reviews the turn for things worth remembering: corrections you made, workflows you walked through, facts about your environment. If it finds something, it proposes a memory save or a skill patch.

This is the part that makes the learning loop feel like magic. You correct the agent once about how your project is structured. The background review catches the correction and saves it. Next session the agent has that fact in context from the first message and you never say it again.

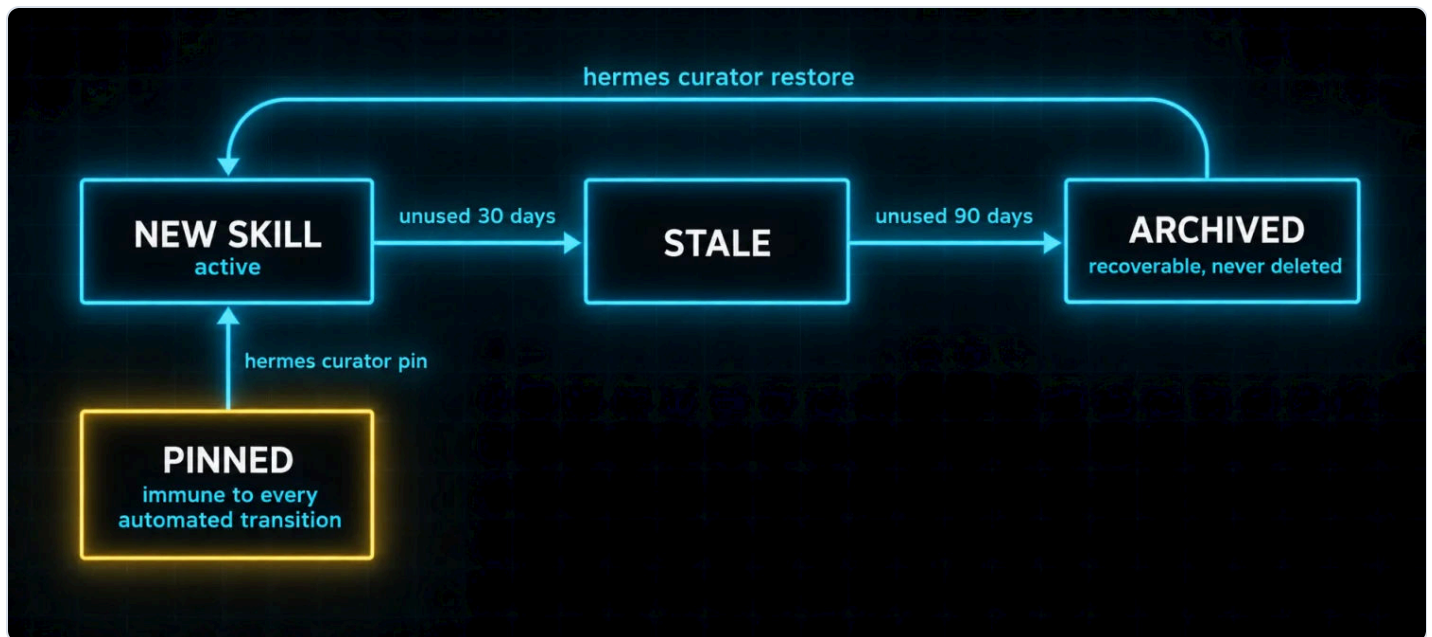
CONTROL	SETTING	EFFECT
Staged writes	memory.write_approval: true	Every proposal is staged, not committed. Review with /memory pending
Staged skills	skills.write_approval: true	Same gate for skill patches. Review with /skills pending
Cheaper reviewer	route the review to a smaller model	Benchmarks showed memory capture identical, skill capture near-identical
Default reviewer	your main chat model	The conversation is warm in its prompt cache, so it is effectively free cache reads

WHY THE DEFAULT IS NOT WASTEFUL

Routing the review to your main model sounds expensive and mostly is not, because the conversation is already in that model's prompt cache. Cache reads are a fraction of fresh input cost. The case for downgrading is when your main model is genuinely expensive per token, and the measured quality cost of doing so was close to zero.

The curator prevents skill rot

The curator is the garbage collector for skills. It runs on a ticker, every 7 days by default, when the agent has been idle for at least 2 hours.



The deterministic phase handles the lifecycle above. **Nothing is ever deleted**, archival is recoverable with `hermes curator restore <name>`.

The optional LLM phase runs an auxiliary-model review: it surveys your library, identifies overlapping skills, proposes umbrella skills that consolidate narrow ones, and patches drift. It is **off by default because it costs tokens**. Opt in with `curator.consolidate: true` or trigger it on demand with `hermes curator run --consolidate`.

Pinning protects critical skills. `hermes curator pin <name>` prevents any automated transition, archival or deletion. Patches and edits still go through, so the agent can improve a pinned skill's content over time, but the skill can never be automatically removed.

Before every curator run, a tar.gz snapshot of the entire skills directory is saved. Any run can be rolled back with `hermes curator rollback`, and the rollback itself is reversible.

How the system compounds

Memory and skills are both injected into the prompt, memory in the volatile tier frozen per session, skills in the stable tier as an always-present index. The agent sees both at all times.

When it encounters a problem it checks memory for relevant facts and loads matching skills. It executes the procedure, calling tools. Results feed back. The background review examines the turn, identifies what worked, and saves it: a memory entry for a discovered fact, a skill patch for a refined procedure.

Over multiple sessions the cycle repeats. Memory consolidates as it fills. Skills are curated as they age. The curator removes what stopped being useful. The agent has fewer, better entries to work with.

This is why the Part 2 decisions matter. The learning loop only works if sessions persist. If every conversation starts from zero, the agent learns in the session, forgets on restart, and never gets better.

OPERATOR DRILL · FORCE A WRITE AND VERIFY IT SURVIVES

Tell the agent a durable, checkable fact about your environment that it could not already know, for example a convention you use in one specific repo. Finish the turn. Start a **new** session and ask it to state that convention back. If it does, the background review wrote it and the volatile tier loaded it. If it does not, check whether `write_approval` is on and the proposal is sitting in `/memory pending`. Either answer teaches you where your learning loop actually stands.

Skills as Executable SOPs



A skill in Hermes is a markdown file. That is it. A plain text file with a YAML header and a body made of headings, bullets and numbered steps. Nothing compiles it. Nothing transpiles it. The agent reads it and follows the instructions.

That simplicity is the point. You do not need a plugin SDK, a manifest file or a deployment pipeline to create a skill. You need a text editor and an opinion about how a workflow should run.

The anatomy of a good skill

Every skill lives at `~/.hermes/skills/<category>/<name>/SKILL.md` and has two parts.

The **frontmatter** carries metadata: name, a one-line description, version, optional platform restrictions, tags and category.

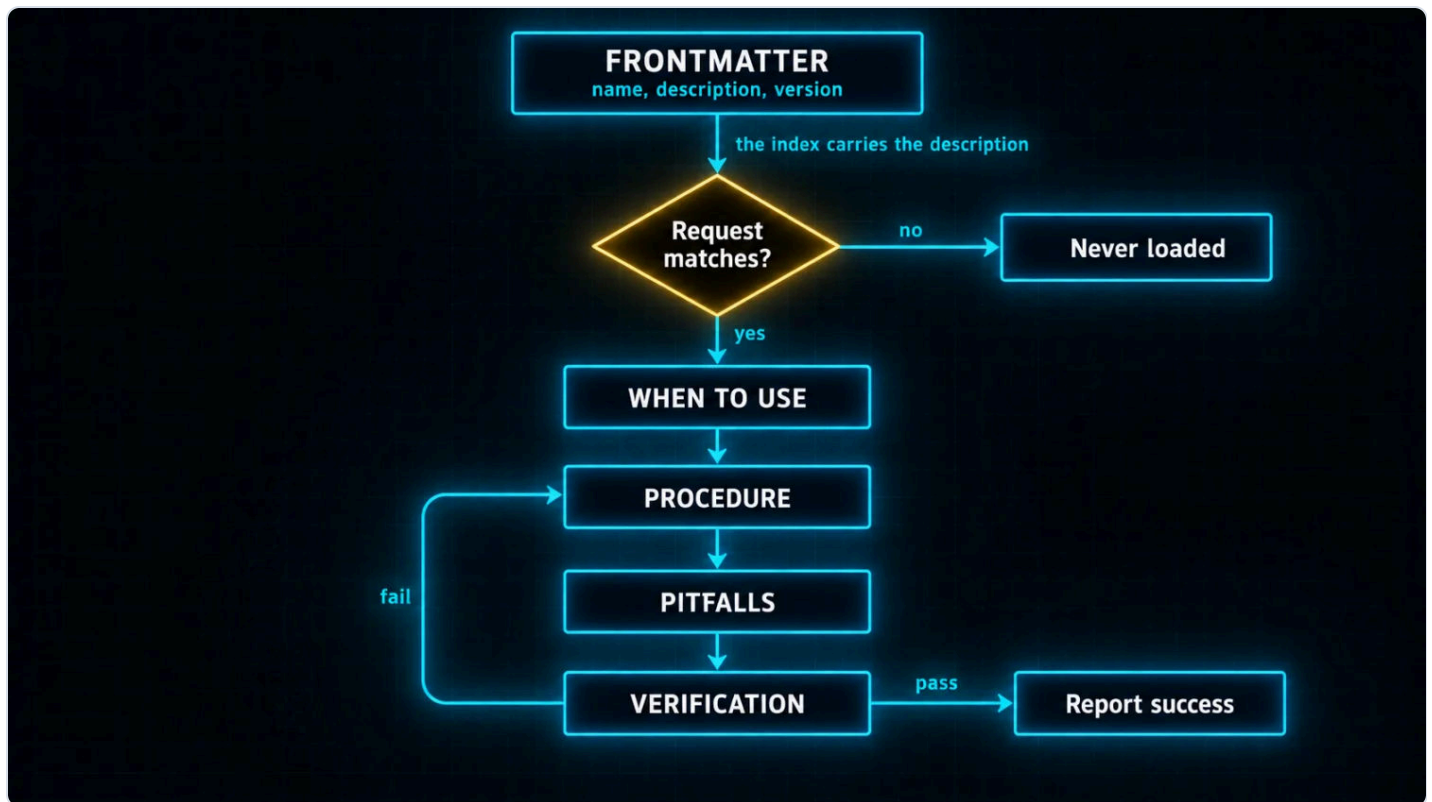
SKILL.md

```
---
name: my-skill
description: Brief description of what this skill does
version: 1.0.0
---
```

THE DESCRIPTION IS THE SINGLE MOST IMPORTANT FIELD

It is what appears in the skill index the agent scans at session start. The body is never read unless the description wins the match. A weak description means the agent never knows to load the skill, and a skill that never loads is a skill that does not exist. Write the description as the trigger condition, not as a summary.

The **body** holds the procedure. The docs recommend four sections, and the four are not arbitrary, each one answers a question the agent would otherwise have to guess.



SECTION	THE QUESTION IT ANSWERS	WHAT A WEAK VERSION LOOKS LIKE
When to Use	Should I load this at all?	"When the user needs help with infrastructure"
Procedure	What exactly do I do, in order?	Prose paragraphs instead of numbered concrete actions
Pitfalls	What will bite me that is not obvious?	Omitted entirely, which is the most common failure

SECTION	THE QUESTION IT ANSWERS	WHAT A WEAK VERSION LOOKS LIKE
Verification	How do I know it worked?	Omitted, so the agent finishes without knowing if it succeeded

A strong "When to Use" is specific: "When the user asks to deploy a service" beats "When the user needs help with infrastructure." A strong Procedure gives one concrete action per numbered step, naming the tool and the arguments when a tool call is required, and naming the decision rule when judgment is required. Pitfalls is where tribal knowledge lives: "The staging server uses port 2222, not 22." "The API returns a 202 before the resource is ready."

Here is the author's worked example, complete.

name: news-research

description: Research a technology news topic -- gather sources, extract claims, produce a brief

version: 1.0.0

When to Use

When the user asks you to research a news story, find the latest information on a topic, or verify a claim from an X post or press release.

Procedure

1. Identify the core claim or announcement. Ask the user to clarify if the topic is vague.
2. Search the web for at least 3 independent sources covering the topic.
3. Extract the key facts from each source: who announced it, what changed, when it happened, and why it matters.
4. Compare the sources. If they agree on the facts, synthesize a summary. If they contradict each other, note the disagreement and why.
5. Present the brief as: two-line summary, key facts in bullets, source links, and any uncertainty you found.
6. Save the research brief as a file at
~/nexus-wiki/wiki/queries/YYYY-MM-DD-topic-slug.md if the user approves.

Pitfalls

- Do not rely on a single source. Three is the minimum for verification.
- Press releases and blog posts are not independent sources. Look for journalism or official documentation.
- If the topic was published more than 6 months ago, flag it as potentially outdated.

Verification

- At least 3 independent sources are cited.
- Every factual claim is mapped to a source URL.
- The brief includes at least one open question or uncertainty.

Notice what the Verification section does: it converts "did the agent do a good job" from a judgment call into three checkable conditions. That is the difference between documentation and an executable SOP.

Progressive disclosure is why skills scale

The agent does not load every skill into every conversation. That would burn tokens on workflows it never uses.

At session start it loads a compact index: every skill's name, description and category, roughly **3,000 tokens for a library of dozens of skills**. When a request matches a description, it calls `skill_view(name)` for the full body. If that skill references supporting files, templates, scripts, reference documents, those load on demand with `skill_view(name, path)`.

Three levels, and the agent pays the token cost only when it actually uses the skill. This is how Hermes ships with dozens of bundled skills and still fits in a reasonable context window.

Stacking, bundling and the ecosystem

Multiple skills run together in one command. Type `/github-pr-workflow /test-driven-development fix issue 123` and the agent loads both files and follows both. The leftmost slash commands are parsed as skill invocations, and parsing stops at the first token that is not a skill name, so path arguments and filenames that happen to start with `/` are never swallowed.

Bundles group related skills under one shortcut. A `backend-dev` bundle might pair code review, testing and PR workflow. Running `/backend-dev` loads all three. Bundles are just YAML files listing skill names, aliases for combinations you use constantly, not replacements for the individual skills.

The Skills Hub is where the community publishes.

```
skills-hub.sh
```

```
hermes skills browse           # see what exists
hermes skills search <keyword> # find by topic
hermes skills inspect <name>   # read it BEFORE installing
hermes skills install <name>   # install, after the security scan
hermes skills tap add <org/repo> # add a team's private GitHub repo of skills
```

The hub covers multiple sources: official optional skills from Hermes, the skills.sh directory from Vercel, well-known endpoints from doc sites, and direct GitHub repos. **Every hub install runs through a security scanner** that checks for data exfiltration, prompt injection and destructive commands.

INSPECT BEFORE INSTALL IS NOT OPTIONAL ADVICE

A skill is instructions a model will follow with your tool surface attached. The security scanner is a real control, but `hermes skills inspect` costs you thirty seconds and lets you read exactly what you are about to give the agent permission to do. Treat an installed skill the way you would treat a shell script from a stranger.

For teams, **taps** let you publish a GitHub repo of SKILL.md files. Members add the tap and install individual skills from it. No registry signup, no server, no pipeline.

What makes skills compound

Two sources feed the library and they compound differently.

The agent creates skills automatically after completing complex tasks. Solve a novel problem with five or more tool calls, and the workflow gets saved. Next time it loads that skill and executes faster.

You create skills for workflows you have already internalized. The deploy runbook. The incident response checklist. The code review standards. The agent does not need to learn these from scratch, it has them as a file it reads and follows.

The curator keeps the library from growing forever, exactly as Part 3 described. The compounding effect is narrower than people expect and that is fine: **every skill makes the agent faster at that specific workflow, not at everything.** A library of 20 well-written skills covering workflows you actually run beats 100 vague ones that never get loaded.

OPERATOR DRILL · WRITE ONE REAL SKILL

Pick a workflow you have explained to the agent more than twice. Write it as a SKILL.md with all four sections, and make the Verification section genuinely checkable. Then start a fresh session and trigger it with a natural request that matches your "When to Use" wording, without naming the skill. If it loads, your description is good. If it does not, your description is a summary rather than a trigger, and that is the most common and most fixable skill-authoring mistake.

Tools and Toolsets



The simplest test for whether Hermes is working is asking it a question and getting an answer. The test works. It is also misleading, because it tests the chat and misses the machine.

What makes Hermes different is the tool surface, not the conversation. The agent runs commands, searches the web, reads and writes files, navigates browsers, generates images, transcribes speech and spawns subagents. **The chat is the input layer. The tools are the output.**

The registry and how tools find their way in

Every tool lives in a central registry at `tools/registry.py`. When a tool module is imported it calls `registry.register()` at module level with its name, schema, handler function and an optional availability check. The call self-registers the tool into a singleton dict keyed by name.

Discovery is automatic. On startup, `discover_builtin_tools()` scans every Python file in `tools/`, does an AST check for top-level `registry.register()` calls, and imports matching modules. **New tool files are picked up without manual wiring.** After the core scan, MCP tools and plugin tools are discovered through their own paths.

The registry holds 70-plus tools across roughly 28 toolsets, and that count excludes MCP tools you connect yourself, which add dynamically.

What the tools actually do

CATEGORY	TOOLS	WHAT IT UNLOCKS
Web	web_search, web_extract	The difference between an agent that makes things up and one that checks its facts
Terminal and files	execute, background process management, read, write, patch edit, content search	Six backends: local, Docker, SSH, Singularity, Modal, Daytona
Browser	navigate, click, type, screenshot, run JS, scroll	Five backends including cloud, local CDP and a managed Chromium
Media	vision analysis, image generation, text to speech	Via the Nous Portal Tool Gateway or individual API keys
Agent orchestration	todo, clarify, execute_code, delegate_task	execute_code collapses multi-step workflows into one inference turn
Memory and recall	memory, session_search	Memory compounds across sessions, search keeps last week reachable
Automation	cronjob	The full lifecycle of recurring jobs from one tool
Integrations	Home Assistant, plus any MCP server	Each MCP server exposes its own toolset under mcp-<server-name>

Toolsets are how you control reach

A toolset is a named bundle of tools, and they exist so you can grant different capability sets to different surfaces. The CLI profile loads a broad toolset. A Telegram bot profile might load messaging, session search and cron, but **not** terminal or browser. The platform preset system handles the common mappings automatically.

```
toolsets.sh
```

```
hermes tools # what is actually loaded right now
hermes chat --toolsets "web,terminal,file" # scope a single session
```

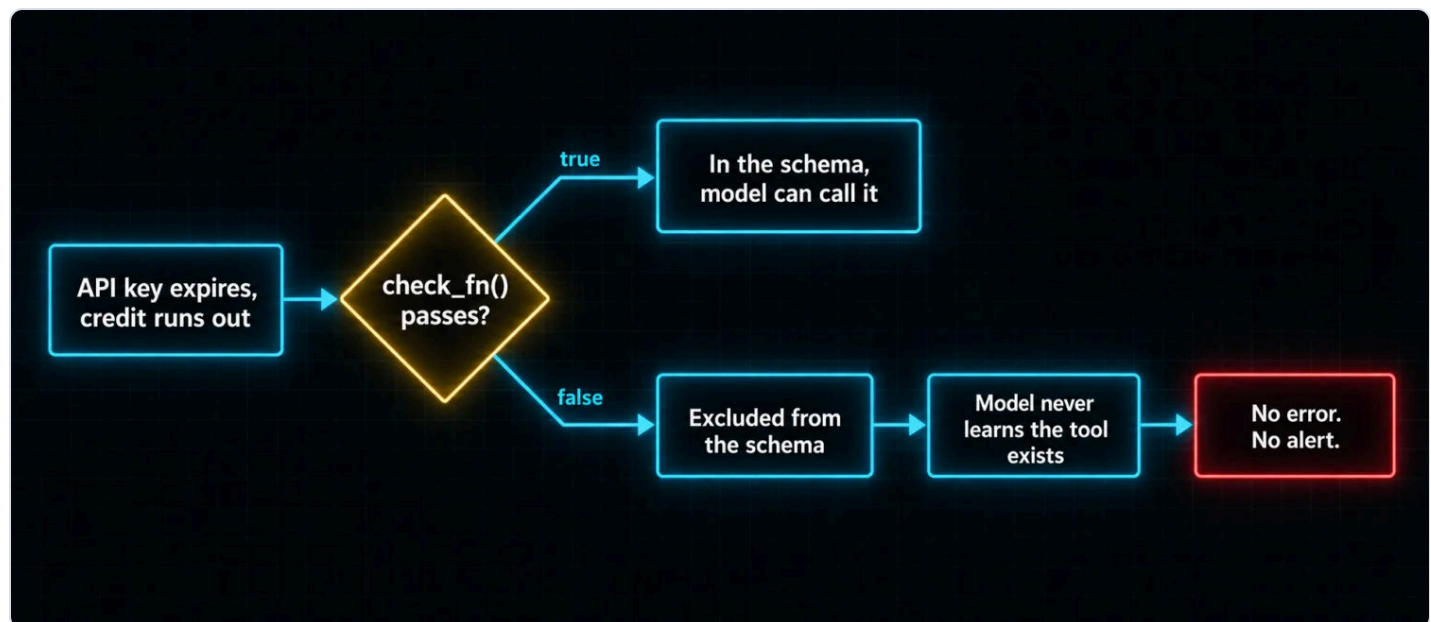
THE 'ALL' SHORTCUT DOES NOT MEAN ALL

`all` enables most toolsets but not every one. Specialist toolsets like `kanban` are opt-in and must be added explicitly alongside `all`. Check `hermes tools` to see what is actually loaded on your current profile before assuming a tool is available. This is the single most confusing behavior in the tool layer.

Availability is dynamic, not static

Every tool can provide a `check_fn`, a callable returning True when the tool can be used and False otherwise. Web search checks for a search API key. The browser tool checks whether a backend is reachable. Image generation checks whether the FAL AI client is installed.

When the agent builds its schema list for the model, it runs each `check_fn` and **excludes unavailable tools from the schema entirely**. The model never sees tool definitions it cannot use.



This cuts both ways and it is worth holding both halves. The good half: **your capability surface changes with your configuration and no code changes**. Install an MCP server, restart, and the agent gains tools. The bad half is Part 12's warning in advance: a credential that expires takes a tool with it, silently, and the agent does not know what it cannot do.

Terminal backends are their own discussion

BACKEND	ISOLATION	PERSISTENCE CHARACTERISTIC
Local	None, full access to your machine	Whatever your machine does
Docker	Read-only root filesystem, dropped capabilities	Single long-lived container; cwd, packages and env persist across tool calls and subagent delegations for the process lifetime
SSH	Delegates to a remote server, keeps the agent away from its own code	Whatever the remote does
Singularity	HPC cluster semantics	Cluster-managed
Modal	Serverless cloud	Hibernates when idle
Daytona	Serverless cloud	Hibernates when idle

The Docker backend deserves attention. Hermes starts one long-lived container on first use and routes every terminal, file and `execute_code` call through it. The container is stopped and removed on shutdown, but with `container_persistent: true` the workspace survives across Hermes restarts.

Background process management is built into the tool surface. The `process` tool can list, poll, wait, kill, write to and close background processes started with `terminal(background=true)`. PTY mode enables interactive CLI tools like Codex and Claude Code, which is how Hermes drives other agents.

Dangerous commands are gated

The terminal tool carries a `DANGEROUS_PATTERNS` regex list covering recursive deletes, filesystem formatting, SQL destructive operations, system config overwrites, service manipulation, remote code execution and fork bombs.

When a command matches, the agent prompts for approval: an interactive prompt in the CLI, an approval request through chat on messaging platforms. **Approvals are tracked per-session**, and a permanent allowlist can be configured in `config.yaml`.

Why the tool surface is the foundation

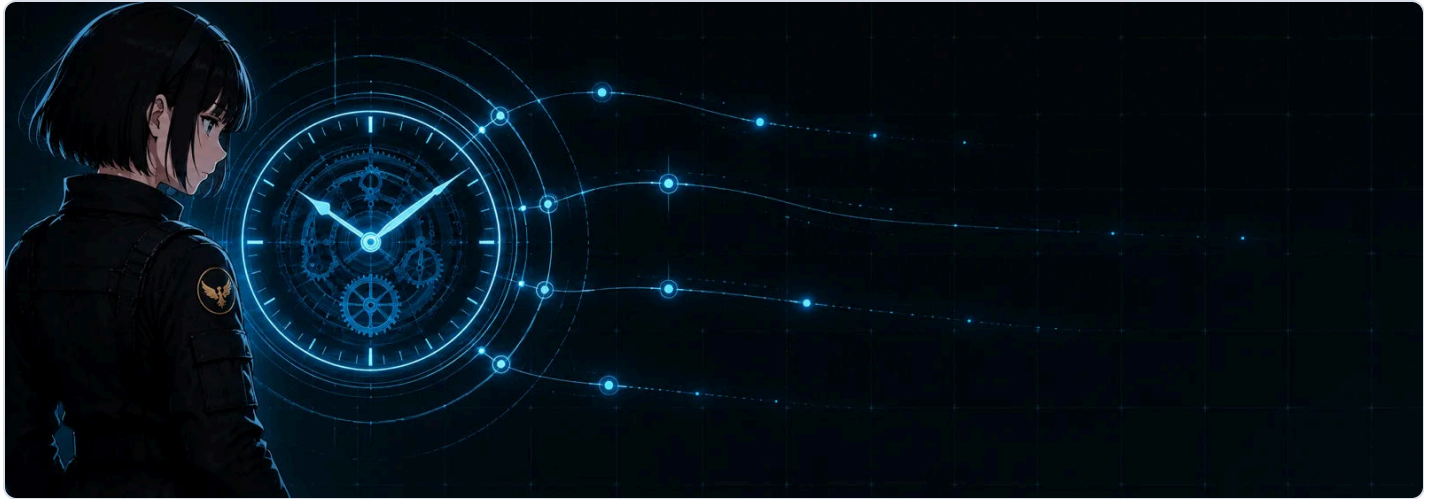
Every feature in the rest of the series depends on tools. Skills are procedures that execute through tools. Cron jobs schedule tool calls. Delegation spawns subagents that use tools. Browser automation and computer use are tool categories. The agent loop dispatches tools.

Without a working tool surface, Hermes is a chatbot with persistent memory. With it, the agent can act on the world.

OPERATOR DRILL · AUDIT YOUR LIVE SURFACE

Run `hermes tools` and read the whole list, not the summary. For every tool you expected to see and do not, find its `check_fn` precondition and fix it. For every tool you see and did not expect, ask whether that surface should be reachable from this profile. Then repeat the exercise on your most locked-down profile, for instance the one behind a messaging gateway. The delta between those two lists is your actual security boundary, and most people have never looked at it.

Cron Makes Hermes Infrastructure

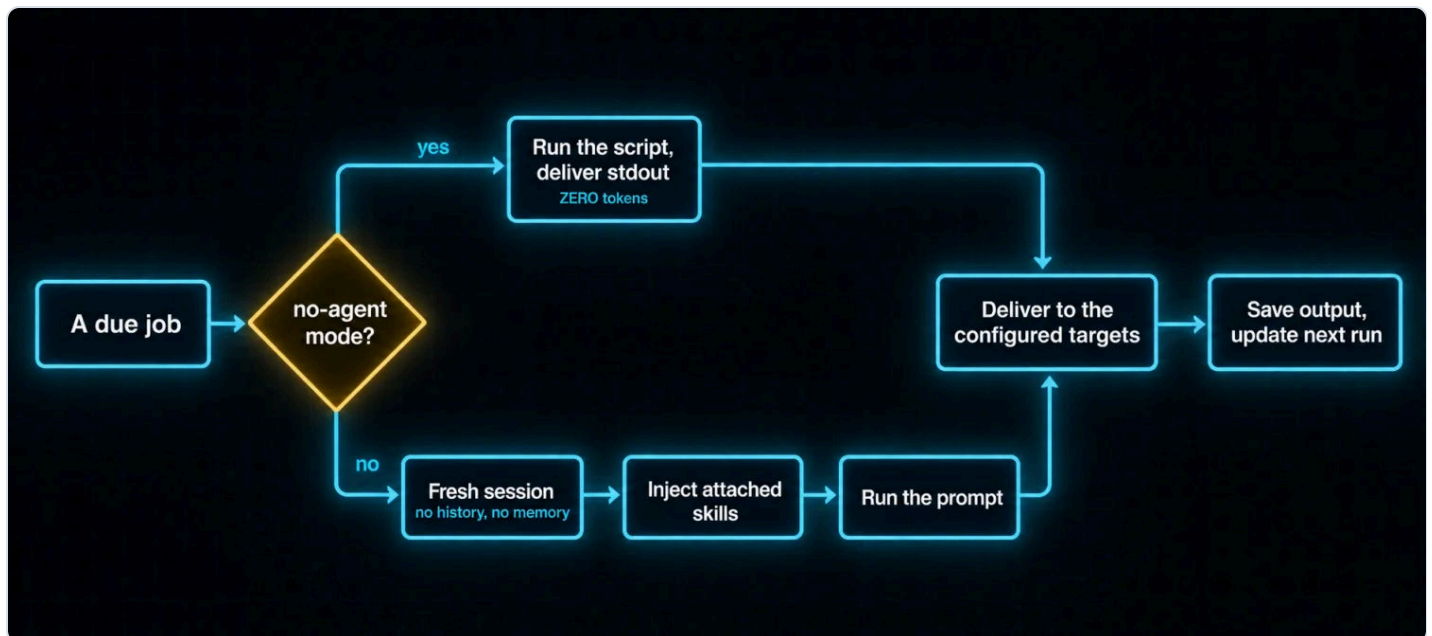
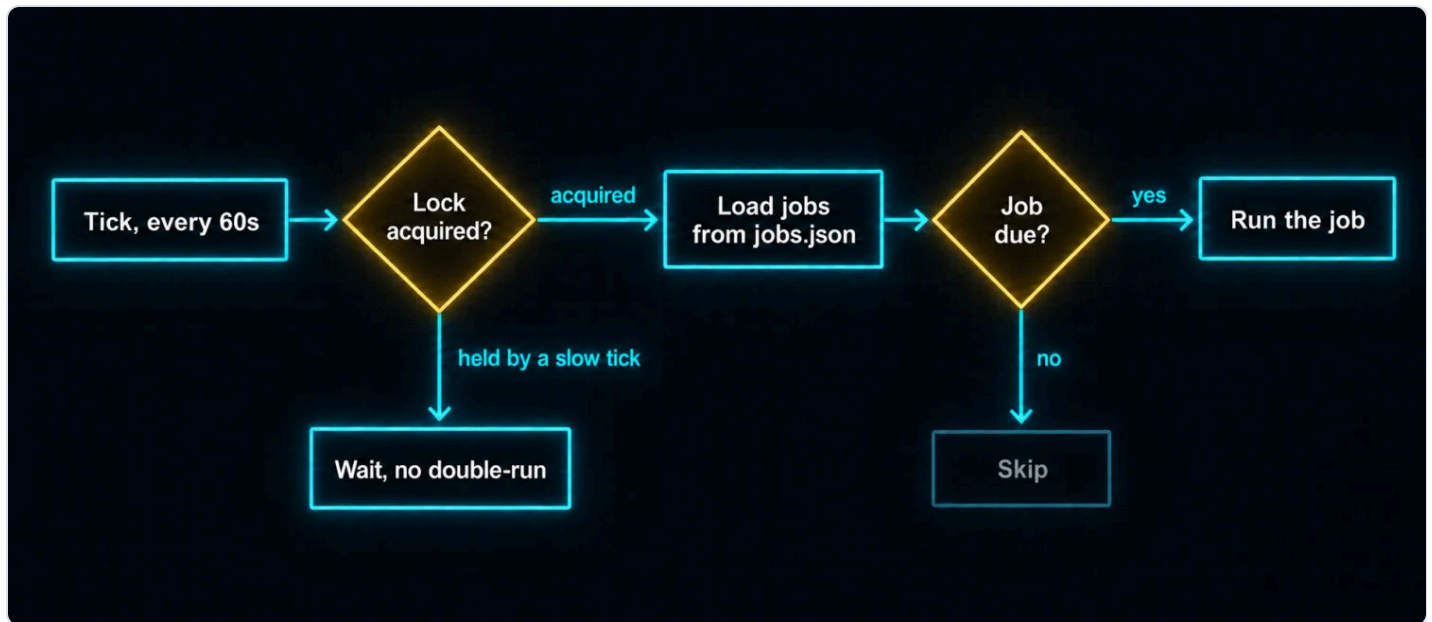


Every part so far has described a reactive system. You send a message, the agent processes it, you get a response. The loop is initiated by you.

Cron breaks that pattern. A scheduled job is something the agent does without being asked, on a schedule you define, in a fresh context every time, delivering results to whatever platform you choose. The agent stops being something you talk to and starts being something that runs.

How cron works under the hood

The cron system lives inside the gateway daemon. Every 60 seconds the scheduler ticks.



The file lock at `~/.hermes/cron/.tick.lock` prevents overlapping ticks from double-running the same batch. If a tick takes longer than 60 seconds, the next one waits. Atomic file writes prevent corrupted job data from a crashed write.

FRESH SESSION MEANS FRESH, WITH NO EXCEPTIONS

No conversation history. No memory from previous runs. A clean slate, every single time. This is the single most important fact about cron and it is the one that breaks people's first three jobs. Everything the agent needs must be in the prompt.

Creating jobs is the best part

You can create a cron job in natural language. "Every morning at 9am, check Hacker News for AI stories and summarize them on Telegram." One sentence. The agent parses the intent, creates the job, and it fires at the next scheduled time.

FORMAT	SYNTAX	BEHAVIOR
Relative delay	"30m", "2h"	One-shot, fires once after the delay
Interval	"every 30m", "every 2h", "every 1d"	Repeats until removed
Cron expression	"0 9 1-5", "0 /6 "	Weekdays at 9 AM; every 6 hours
ISO timestamp	"2026-07-15T09:00:00"	Exact date-and-time one-shot

The full lifecycle runs through a single `cronjob` tool: create, list, pause, resume, run now, edit, remove. All of it from a chat message, no CLI required. The agent also accepts **job names in place of hex IDs**, so "pause the morning briefing" works instead of copying an ID.

Skills make cron smarter

A cron job can load one or more skills before running its prompt. This is the difference between a scheduled task that fumbles through a workflow and one that executes a known procedure.

A single attached skill behaves like a reusable playbook: its trigger conditions, numbered procedure, pitfalls and verification become the job's operating context. Multiple skills load in order, so a morning digest job might load `blogwatcher` and `maps` together and have both contexts available.

THE PROFILE TRAP THAT COSTS PEOPLE AN AFTERNOON

Skills attached to cron jobs must be available **in the profile that runs the gateway**. Cron jobs run in the gateway process, and the gateway uses the default profile's skill catalog. If your skill lives in a sub-profile, the job will run and quietly behave as though the skill does not exist. Copy it to the default profile's skills directory.

Delivery is the whole point

A cron job that runs but never tells you the result is a job that did not happen. Delivery covers 20-plus platforms.

TARGET	MEANING
origin	Back to where the job was created. The default for messaging platforms
local	Saves output to files at ~/.hermes/cron/output/. The default for CLI-created jobs
all	Every connected home channel, resolved AT FIRE TIME
"telegram,discord"	Exactly those two
"origin,all"	The origin chat plus every other channel

The `all` token resolving at fire time is a small detail with a real payoff: **a job created before you wired up Discord picks up Discord automatically once you configure it.** You do not go back and edit old jobs.

Two behaviors are worth knowing by name.

The continuable flag. By default cron is fire-and-forget: the message lands but you cannot reply to it with the agent aware of context. With `attach_to_session: true`, the delivery is seeded into a session thread. You reply and the agent has the brief in context. On thread-capable platforms like Telegram and Discord, each delivery opens its own dedicated thread.

The silent suppression pattern. If the agent's final response contains `[SILENT]`, delivery is suppressed entirely. The output is still saved locally for audit, but no message is sent. This is *the* pattern for monitoring jobs:

```
monitoring-job-prompt.txt
```

```
Check if nginx is running. If everything is healthy, respond with only [SILENT].  
Otherwise, report the issue.
```

No-agent mode is the hidden gem

Not every scheduled job needs an LLM. A memory usage check. A disk space alert. A heartbeat ping. These are classic watchdog patterns that should be cheap and reliable.

No-agent mode skips the LLM entirely. The scheduler runs your script on schedule and delivers its stdout directly. **Zero tokens. Zero provider calls. Zero model fallback.**

SCRIPT OUTCOME	WHAT HAPPENS
Non-empty stdout	Delivered verbatim
Empty stdout	Silent tick. Nothing sent, nothing logged as a failure
Non-zero exit code	Error alert fires
Timeout	Error alert fires

That last pair matters: **a broken watchdog cannot fail silently**, which is the classic way monitoring lies to you.

The agent can set these up for you. Describe the watchdog in chat, "Ping me on Telegram if RAM is over 85%, every 5 minutes", and Hermes writes the check script, creates the no-agent job, and wires the delivery. The entire lifecycle happens without touching the CLI.

Chaining jobs creates pipelines

Cron jobs run in isolated sessions with no memory of previous runs. But sometimes one job's output is exactly what the next needs.



The `context_from` parameter wires the connection automatically: Job B gets Job A's most recent output prepended as context at runtime. The chain can be any length, and each job fires on its own schedule reading the upstream job's last output.

The `wakeAgent` gate lets a pre-check script decide at runtime whether the LLM should be invoked at all. A script that polls a feed, an API or a database emits `{"wakeAgent": false}` if nothing changed, skipping the agent turn. For frequent polls every 1 to 5 minutes, this is the difference between paying for zero-content agent turns and paying for nothing.

Self-contained prompts are non-negotiable

Because cron runs in a completely fresh session, the prompt must contain everything.

THIS FAILS

THIS WORKS

"Check on that server issue"

"SSH into server 192.168.1.100 as user deploy, check if nginx is running with `systemctl status nginx`, and verify `https://example.com` returns HTTP 200"

The first fails because the agent has no idea what server issue you mean. The second works because every detail is in the prompt. If the task changes, edit the prompt with cron's edit action rather than deleting and recreating.

What cron changes

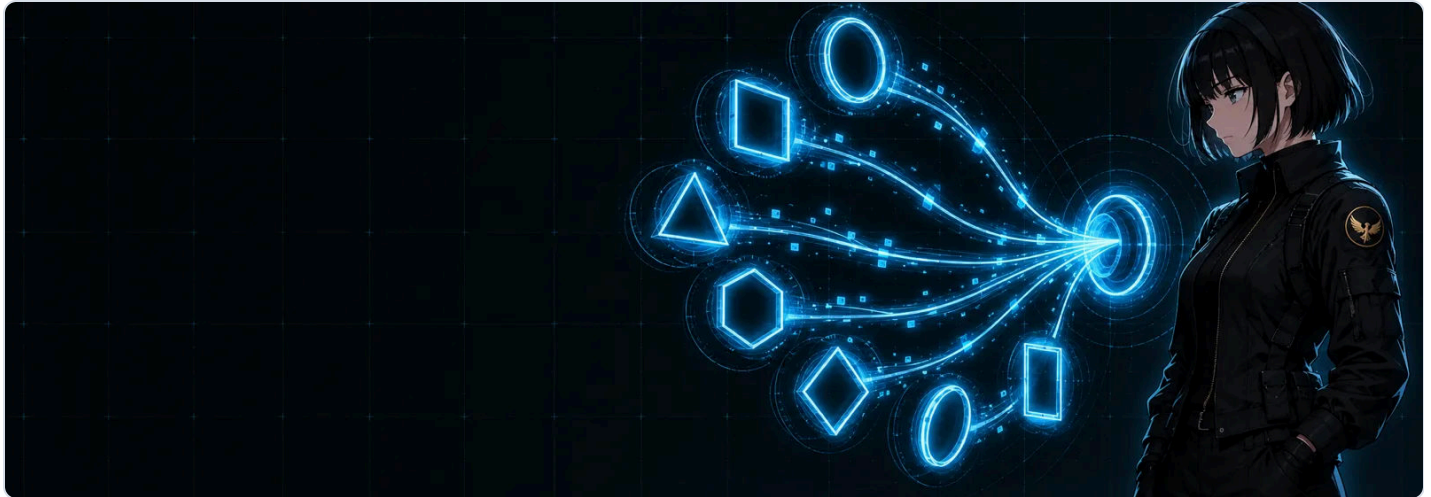
Before cron, Hermes is a tool you reach for. Reactive by definition. After cron, Hermes runs on your behalf: it checks things you would forget to check, summarizes things you would miss, and delivers to the same chat you use for everything else.

The transition from reactive to proactive is what makes Hermes feel less like a chatbot and more like an operator.

OPERATOR DRILL · SHIP A ZERO-TOKEN WATCHDOG

Create a no-agent cron job that checks one real condition on your machine every 5 minutes and prints nothing when healthy. Then deliberately break the condition and confirm the alert arrives. You have now built monitoring that costs zero tokens, cannot lie to you when the script dies, and took one chat message to create. Do this before you build any LLM-backed scheduled job, because most of what people reach for cron to do does not need a model at all.

Messaging Gateways Make Hermes Ambient

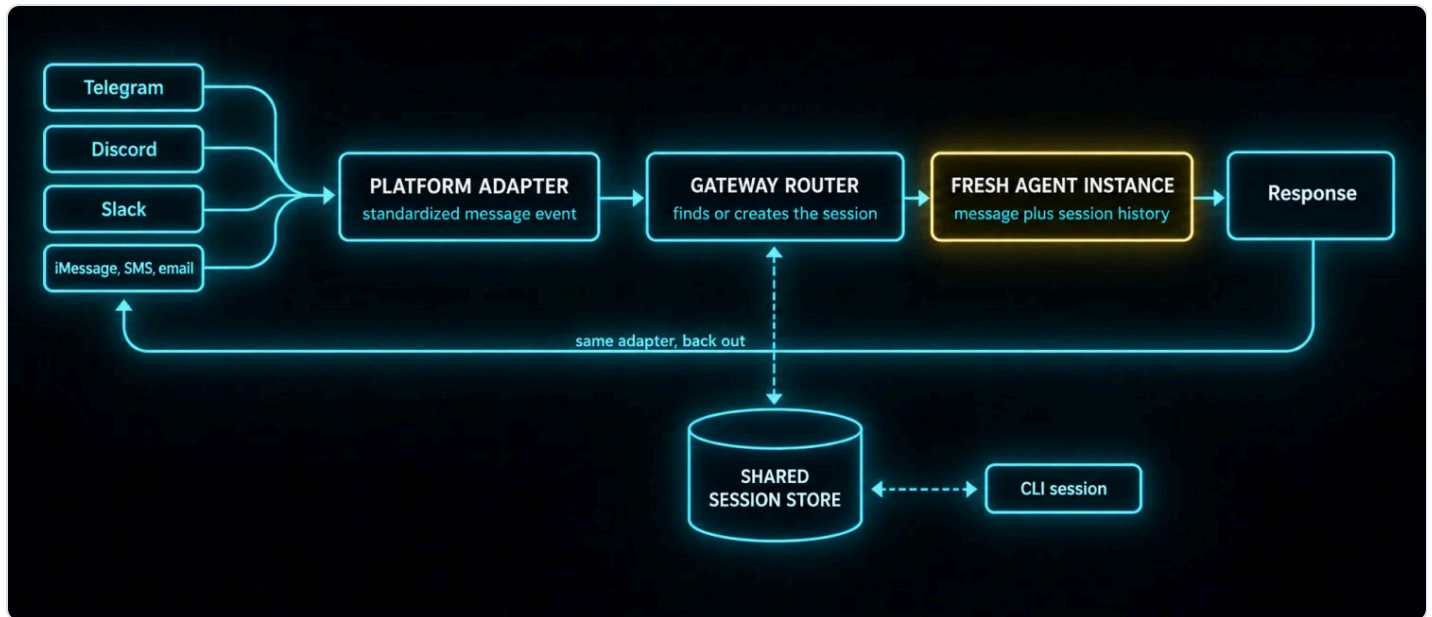


Everything so far has assumed you are sitting at a keyboard. CLI, TUI, desktop app. It works, and it is also a limitation, because it means Hermes is only useful when you are at your machine.

Messaging gateways break that constraint. Telegram, Discord, Slack, WhatsApp, Signal, iMessage, SMS, email, Matrix, Teams and about a dozen more. **The agent does not know or care which platform you are using.** Sessions carry across platform boundaries: start something in the CLI, check the result on Telegram, pick it back up in Discord.

The gateway is a single background process. It runs alongside the agent, connects to every platform you configure, handles session routing, runs cron jobs and delivers voice messages. It is what makes Hermes feel like infrastructure instead of an application.

How the gateway works



Sessions are portable because the session store is shared. A session started on Telegram has the same history and memory as one started in the CLI or on Discord. You can start a research task on your phone, walk to your desk, and see the same conversation in the terminal.

Setup is one command

```
gateway-setup.sh
```

```
hermes gateway setup
```

The interactive wizard walks through each platform with arrow-key selection, shows which are already configured, validates credentials, and offers to start the gateway when done.

Telegram is the easiest path for most people. Create a bot through BotFather, paste the token, done. No static IP required, no webhook configuration. The gateway uses long polling, so it works behind NAT, on a laptop that changes networks, or on a VPS with a dynamic IP.

Discord needs a bot token and application ID from the Developer Portal. WhatsApp uses the built-in Baileys bridge. Signal needs a signal-cli daemon.

Platform personality

Different platforms get different tool configurations by default, and this matters more than you would think. A five-paragraph response with markdown tables looks great in the CLI and terrible on a phone screen.

SURFACE	DEFAULT POSTURE
Telegram	Treated as a mobile inbox. Auto-progress messages off to reduce notification noise, shorter responses, plain text preferred over markdown
CLI	Broad toolset including terminal and file access
Discord	In the middle

The platform hint system tells the agent where it is talking and the agent adjusts output format accordingly. You can override the defaults per platform: tool-progress messages, still-working heartbeats and status updates are independently switchable, and the `platform_hints` config key lets you append or replace the per-platform guidance the agent receives.

Authorization keeps it secure

By default the gateway denies every message from every user who is not explicitly authorized. Nobody can message your agent unless you approve them.

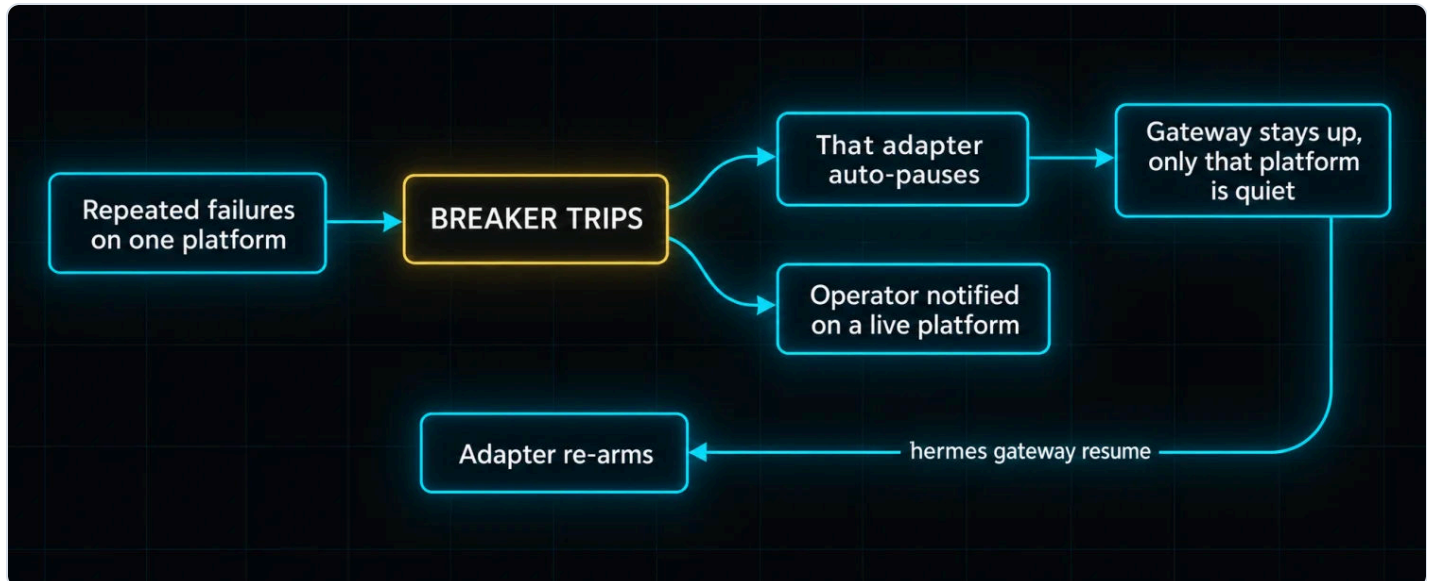
MECHANISM	HOW IT WORKS
Allowlist	A list of user IDs permitted to talk to the agent
DM pairing	A one-time code the user types into a direct message to prove they control the account
Admin tier	Full access, all slash commands, can manage the gateway
User tier	Standard conversation access

Tiers are configured independently per platform, and **DM admin does not imply group admin.** When an unpaired user sends a message the agent ignores it unless pairing is

initiated from an authorized account.

Circuit breakers keep it running

Every platform adapter is wrapped in a circuit breaker. Repeated retryable failures, network blips, rate-limit replies, 5xx responses, websocket disconnects, trip the breaker.



The `/platform` slash command lets you inspect and steer individual adapters without restarting the gateway: list all, pause one, resume one. **Pausing keeps the adapter loaded and its background loops alive** so incoming messages are silently dropped but the connection stays open, which makes resume instant.

The ambient shift

The gateway turns Hermes from something you open when you need it into something that is just there. You message it from your phone while commuting and continue the same conversation at your desk. Cron results and alerts land in the same chat you already use.

When a tool call takes time the gateway pushes progress updates. When it restarts after an update it sends a one-shot notification to each platform's home channel so you know it is back.

And it closes a loop from Part 3: **the learning system only compounds if the agent is reachable**. A skill you wrote is useless if you can only reach it from a terminal you are not

sitting at.

OPERATOR DRILL · CROSS-SURFACE ONE SESSION

Start a task in the CLI that produces a durable intermediate result. Without finishing it, open Telegram and ask the agent what it was just working on. If the session store is genuinely shared, it answers from the CLI context. Then finish the task from the phone and confirm the result is visible back in the terminal. This single exercise proves session portability, gateway routing and the shared store in about two minutes.

Delegation and Subagents



A single agent session is a serial process. One message in, the model thinks, calls tools, evaluates results and responds. Everything in sequence. For most tasks this is fine. For research, code review, refactoring and parallelizable workflows, it is a bottleneck.

Delegation spawns child agents that work independently and in parallel. Each child gets its own conversation, its own terminal session, its own toolset and its own budget. **Only the final summary comes back to the parent.** The children's intermediate work, their tool calls, errors, dead ends and debugging noise, stays in their context and never enters yours.

That last clause is the real product. Delegation is as much a context-management technique as a speed technique.

How delegation works

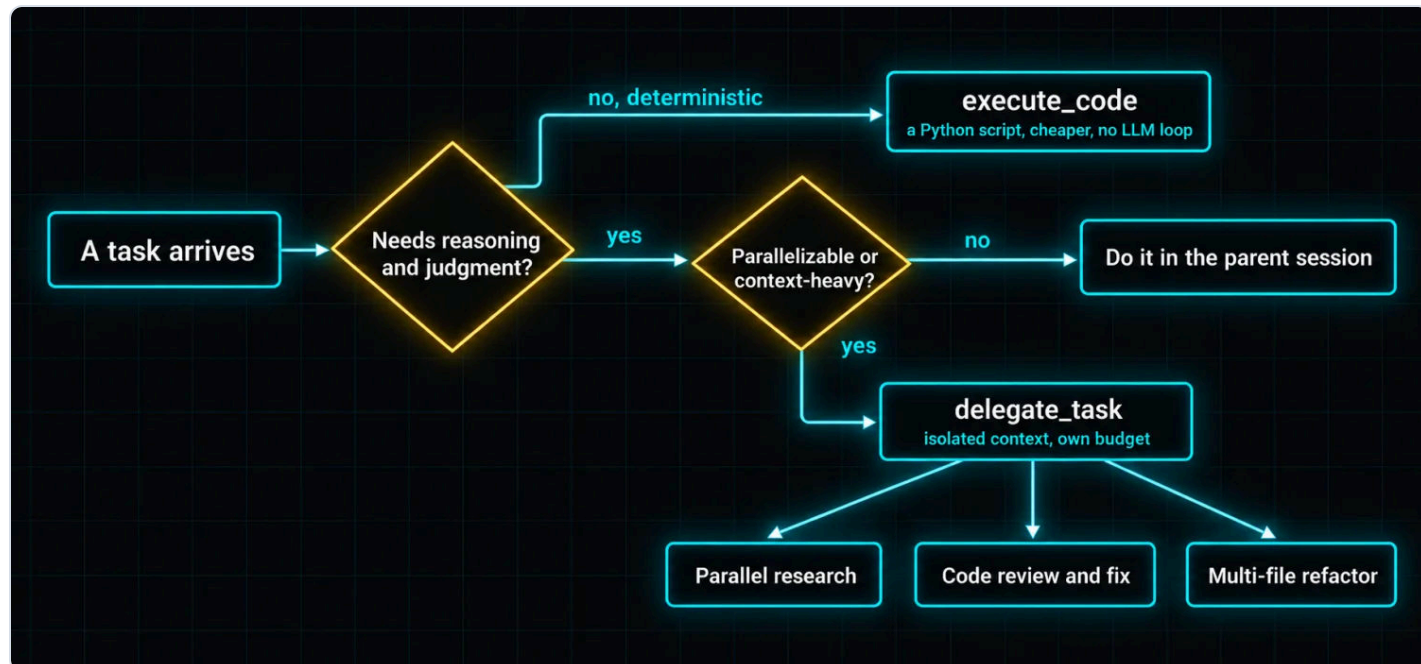
The `delegate_task` tool spawns a fresh `AIAgent` instance. The child starts with a completely clean conversation and zero knowledge of the parent's history. The only context it receives is what the parent puts into the `goal` and `context` fields.

SUBAGENTS KNOW NOTHING. THIS IS THE WHOLE LESSON

If you delegate a task and say "fix the error," the subagent has no idea what error you mean. The goal must be self-contained: file paths, error messages, project conventions, expected outcomes, all of it in the goal and context fields. This is the same constraint as a cron prompt from Part 6, arising from the same cause, a fresh session with no history.

A single task creates one child. A batch creates up to three by default, running concurrently in a thread pool. The parent blocks until all children complete, and results are returned in input order regardless of completion order.

When to delegate, and when not to



The three patterns the author calls out, with what each one actually buys:

Parallel research. Three subagents research three topics simultaneously, each running independent web searches and returning a structured brief. The parent synthesizes. This is the highest-ROI use case for most people.

Code review and fix. Delegate a security audit of the authentication module. The child reviews, finds issues, fixes them, runs tests and returns a summary of what it changed. The parent never sees the debugging process.

Multi-file refactoring. A refactor touching 20 files generates enormous intermediate context. Delegating keeps the parent session clean. The child works through each file and returns a summary; the parent only loads the diff.

Toolset selection is a cost lever, not just a permission

Each subagent gets its own `toolset` parameter. Research children need `["web"]`. Code children need `["terminal", "file"]`. Full-stack tasks get `["terminal", "file", "web"]`.

TOOL	STATUS FOR SUBAGENTS	WHY
clarify	Blocked	Subagents cannot interact with the user
memory	Blocked	Subagents should not write to shared persistent state
delegation	Blocked for leaf subagents	Prevents runaway recursive spawning

Restricting the toolset also **reduces token consumption**. A research child with only web tools does not load terminal, file, browser or image tool schemas into its prompt. The model has fewer options to consider and a smaller tool index. Scoping a child is therefore both a safety decision and a cost decision, and most people only think of it as the former.

The async model

The original delegation tool blocked the parent chat while children ran. You fired three research subagents and sat watching a spinner. If a child got stuck you either waited it out or cancelled the whole batch.

Async subagents fixed this. `delegate_task_async` fires a subagent and returns immediately. You keep working.

ACTION	WHAT IT DOES
check	Progress on running children
steer	Redirect a stuck child mid-execution

ACTION	WHAT IT DOES
collect	Gather results once they are done
cancel	Kill tasks no longer needed
list	See the full fan-out

This is the model that makes delegation feel like real parallelism instead of slower sequential work. **Fire and forget. Check later. Collect when ready.**

The `/agents` slash command in the TUI turns the fan-out into a live tree view: running children, finished results, per-branch cost and token rollups, kill and pause controls, and turn-by-turn history. The classic CLI prints a text summary; the TUI renders it as an interactive overlay.

Budgets, timeouts and depth

CONTROL	DEFAULT	NOTES
Child iteration limit	50 turns	A file check might take 5; a deep review might need all 50
Child timeout	Removed by default	An earlier hard 300s cap killed legitimate long-running work
Stuck detection	Heartbeat staleness monitor	Replaces the removed timeout
Hard timeout	Opt-in via config	For cost control on unattended cron-driven delegation
Concurrent children per batch	3	The default fan-out width
Orchestration depth	1 (flat)	Raise to 2 to allow one level of nesting

DEPTH MULTIPLIES, AND IT MULTIPLIES FAST

By default delegation is flat: a parent spawns children, and those children cannot spawn their own. For multi-stage workflows an orchestrator child retains the delegation toolset and can spawn leaf workers. Each level multiplies cost. Depth 3 with 3 concurrent children means **27 leaf agents** at the deepest point. Raise the depth limit intentionally, never casually.

When parallelism changes the workflow

Single-agent sessions are simple, predictable and easy to debug. You see every tool call and know what the agent is doing.

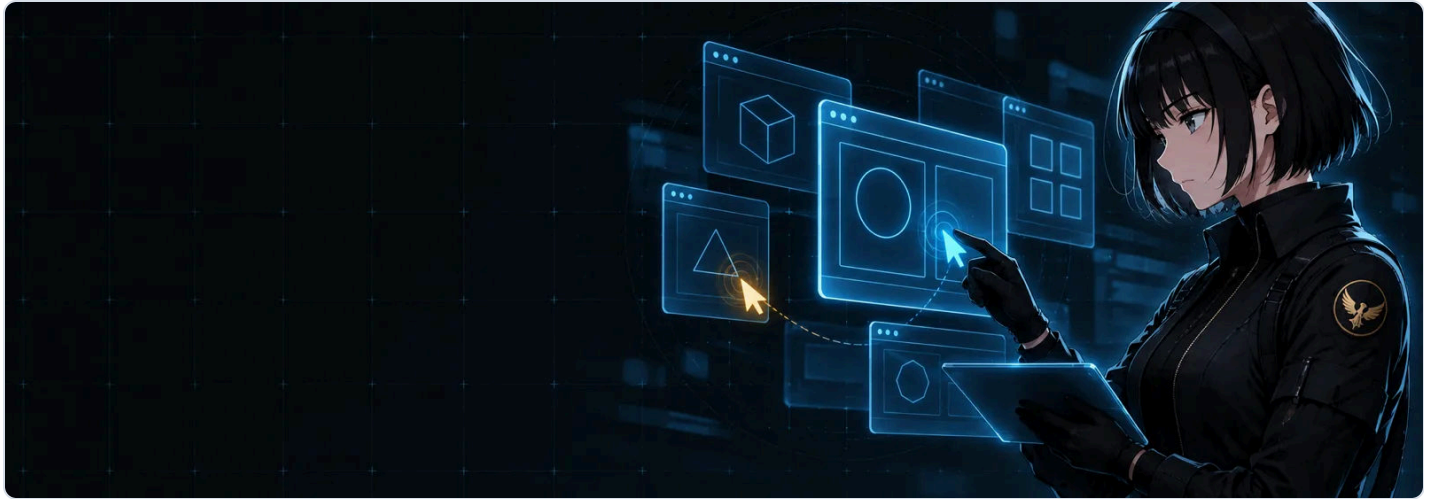
Parallel delegation introduces uncertainty. Three subagents run simultaneously, you do not see their intermediate work unless you check, one might get stuck, and in the synchronous model the parent is blocked until all complete.

The tradeoff is worth it when the parallelism saves meaningful time. Three research tasks in sequence take three times as long as one; in parallel they take roughly as long as the slowest. For comparative work, testing three approaches, researching three vendors, reviewing three modules, delegation turns a 15-minute wait into a 5-minute wait. The async model eliminates the waiting entirely.

OPERATOR DRILL · A REAL THREE-WAY FAN-OUT

Pick a genuine comparison you need to make, three tools, three vendors, three approaches. Fire three async subagents, each scoped to `["web"]` only, each with a fully self-contained goal naming exactly what to find and what shape to return it in. Keep working while they run. Collect, then have the parent synthesize. Two things to notice afterwards: how little of their intermediate noise reached your context, and whether any child failed because your goal assumed knowledge it did not have. The second is the lesson.

Browser and Computer Use



Every tool so far has been text-based. The terminal runs commands and returns text. File tools read and write text. Web search returns text. Even cron is text input to text output.

Browser automation and computer use break that pattern. The browser tool navigates real websites, clicks real buttons, fills real forms. The computer use tool drives the actual desktop, clicking, typing, scrolling, dragging, on macOS, Windows and Linux. **Your cursor does not move. Your focus does not shift.** The agent works alongside you on the same machine.

These are the tools that let Hermes operate in interfaces that were never designed for APIs.

Browser automation

The browser toolset turns Hermes into a real web browser: navigate, click, type, screenshot, run JavaScript, scroll. The agent sees the page as an **accessibility tree**, a text-based snapshot with numbered ref IDs for every interactive element. It clicks `e5` to press a button, types into `e3` for a search field, and reads the console for JavaScript errors.

BACKEND

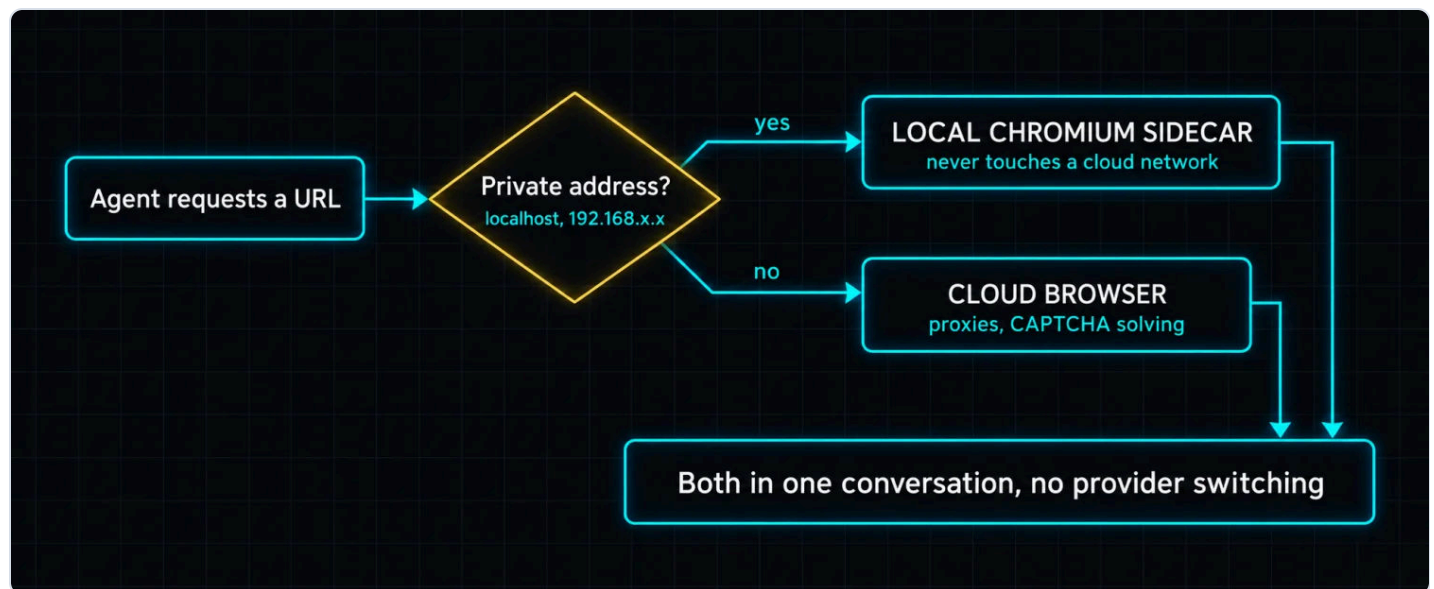
WHAT IT IS FOR

Browserbase cloud

Managed cloud browsers with residential proxies and CAPTCHA solving. For sites that fight bots

Browser Use cloud	Alternative cloud provider with its own anti-detection
Firecrawl	Another cloud provider option
Local Chromium via CDP	Attaches to your own Chrome or Brave through DevTools Protocol
Managed local Chromium	The default, driven by the agent-browser CLI

Hybrid routing is the feature worth calling out. With a cloud provider configured, public URLs go through the cloud browser while `localhost`, `192.168.x.x` and other private addresses automatically route to a local Chromium sidecar.



The payoff: the agent can screenshot `http://localhost:3000` and scrape `https://github.com` in the same conversation, and your local dev server never leaves your machine.

The canonical demonstration is a web form. The user says "sign up for an account." The agent navigates to the signup page, takes a snapshot, sees the form fields with ref IDs, types email and password into the right inputs, clicks the create-account button, and takes another snapshot to confirm success. That interaction was impossible for text-only agents.

Computer use

Computer use extends the same principle to the entire desktop. The agent captures any visible window as a screenshot with **numbered overlays on every interactive element**, then clicks by element index rather than pixel coordinates, which is dramatically more reliable. It types text, presses key combos, scrolls and drags.

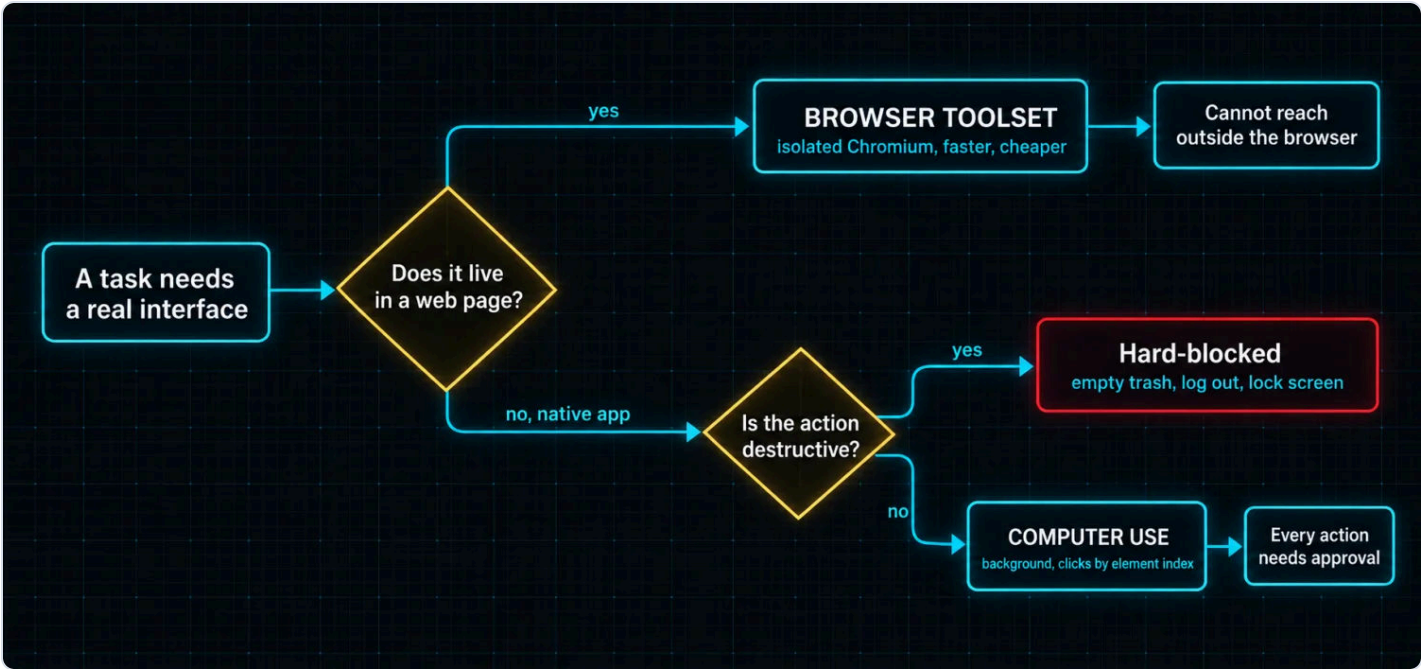
The background execution model is the key design choice. When the agent clicks something, your real OS cursor stays where it is. The window it is operating on never comes to front. Virtual desktops do not switch. A tinted overlay cursor shows where the agent is acting so you can see what it is doing without losing your place.

The agent cursor is **session-scoped**: each Hermes session and each subagent gets its own cursor identity, so concurrent work does not produce confusing double-cursor behavior.

Computer use works with any tool-capable model, Claude, GPT, Gemini or an open model on a local endpoint. There is no vendor-specific schema. The toolset speaks MCP over stdio to `cua-driver`, an open-source background driver handling the platform-specific accessibility stack.

Choosing between them

DIMENSION	BROWSER TOOLSET	COMPUTER USE
Operates on	An isolated Chromium instance	Your actual desktop
Can accidentally	Nothing outside the browser session	Open apps, change system settings
Reaches	Websites, with their own cookies, cache, fingerprint	Native apps and dialogs with no web equivalent
Cost	Faster, cheaper	Screenshot-heavy, more expensive
Permissions	None platform-specific	Platform accessibility grants required
Use it when	The task is web-only	The app has no web interface



The rule is simple: **for web-only tasks use the browser toolset**, it is faster, cheaper, isolated and needs no platform permissions. For native desktop tasks use computer use.

Safety and guardrails

The browser toolset has no dangerous-command concerns because everything runs inside an isolated session. Its main limitations are what it cannot do: it cannot download files from the browser, it relies on the accessibility tree rather than pixel coordinates, and sessions expire based on your provider's plan.

Computer use carries a more extensive safety model.

GUARDRAIL	BEHAVIOR
Permission dialogs	Every capture showing one is flagged
Hard-blocked actions	Empty trash, log out, lock screen, force delete
Filtered key combos	Windows key and similar

Dangerous shell patterns	Blocked at typing time
Screenshots as data	The agent is told not to follow directives embedded in screenshots, preventing prompt injection via UI
Approval gating	You see every action before it executes. Interactive prompt in CLI, approval buttons on messaging platforms. Manual mode in config requires confirmation for every action

SCREENSHOTS ARE DATA, NOT INSTRUCTIONS

This is a genuinely important control and it is easy to skim past. A screenshot can contain text. Text can contain instructions. Without an explicit rule, an agent looking at a malicious web page or a crafted document could read "ignore previous instructions" off the pixels and comply. Hermes tells the model that screenshot content is data to be described, never directives to be followed. That is prompt-injection defense at the UI layer.

Token efficiency

Screenshots are expensive. A single 1568x900 screenshot costs roughly **1,500 tokens**. A 20-action session without optimization would burn through context in minutes.

OPTIMIZATION	EFFECT
Recent-three window	The adapter keeps only the three most recent screenshots in context; older become placeholder text
Context compressor	Strips old image parts from tool results
Flat-rate counting	Each image counts at Anthropic's flat 1,500 tokens regardless of base64 length
Server-side clearing	On Anthropic, old tool results are cleared server-side

The measured result: **a full session typically costs around 30K tokens of screenshot context instead of 600K**. That is a 20x reduction and it is the difference between computer use being usable and being a novelty.

The **accessibility-tree mode** is the fallback for text-only models or when you want to save tokens entirely. The agent gets the structured tree without the screenshot. It can still navigate, click and type, it just cannot see visual layout.

OPERATOR DRILL · PROVE HYBRID ROUTING

Start a local dev server on port 3000. In one conversation, ask the agent to screenshot `http://localhost:3000` and then extract something from a public site. Check your cloud browser provider's session log afterwards. You should see exactly one session, for the public URL. If your localhost request appears in the cloud provider's logs, hybrid routing is not configured and you have been shipping your local environment to a third party.

Kanban as a Coordination Model



Everything so far has assumed one agent, one conversation, one task at a time. You ask, the agent does, you get a result. Serial processing.

Kanban breaks that model. It gives you a durable, SQLite-backed task board that multiple agents can read from, claim and write to. The board survives restarts. Tasks have states, assignees, priority and tags. **The agent does not need to remember what work is open, it reads the board.**

Kanban in Hermes is not a project management feature. It is an agent coordination model for work spanning multiple profiles, sessions and agent instances.

The board model

A kanban workspace is a SQLite database at `~/.hermes/kanban/`. It stores tasks with labels, states, assignees, priority, tags and timestamps, and the agent interacts with it through the `kanban` tool.

STATE

MEANING

todo

Created, unclaimed

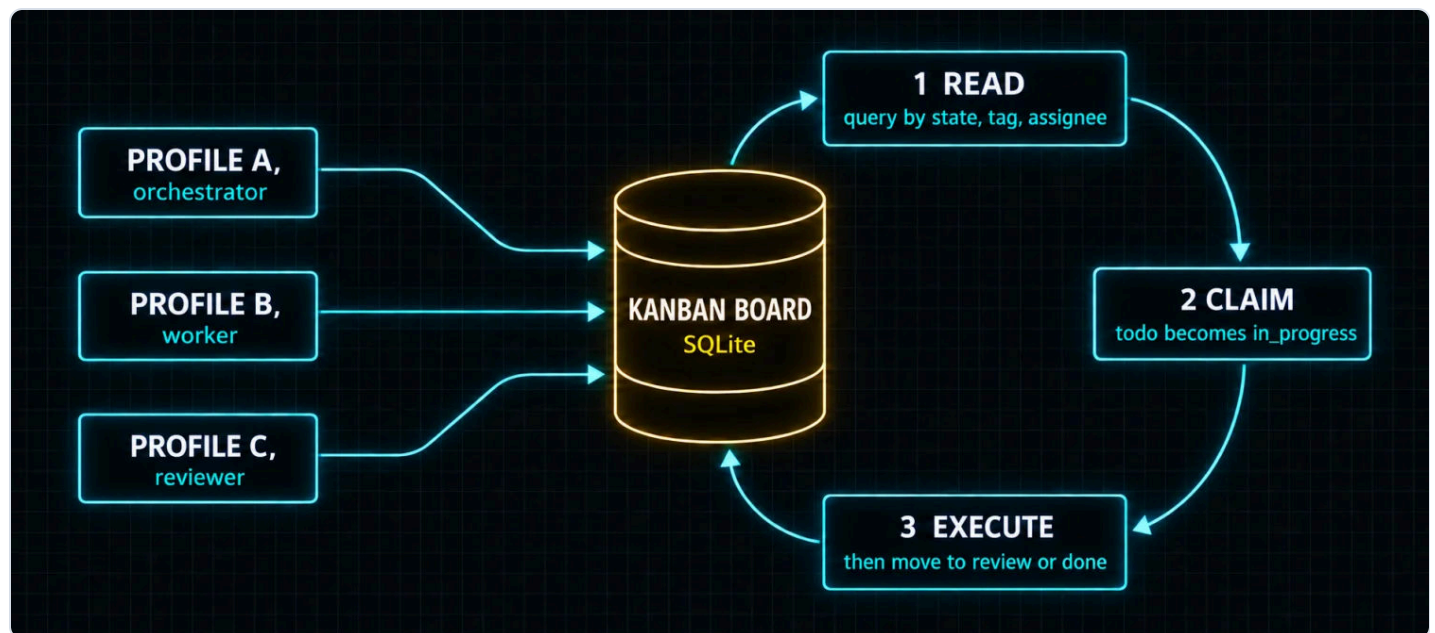
in_progress

A profile has claimed it and is working

STATE	MEANING
review	Work is done and awaiting validation
done	Finished
cancelled	No longer makes sense

Each task has an **assignee field that maps to a Hermes profile**. When a profile picks up a task it claims ownership, and other profiles can see who is working on what and avoid duplicating effort. Priority ranks tasks within a state; tags group related tasks. The agent filters by state, priority, tags or assignee to find exactly the work it should be doing.

Read, claim, execute



Read. The agent queries the board for tasks in a relevant state. An orchestrator might query all `todo` tasks tagged "research"; a worker might query `in_progress` tasks assigned to itself.

Claim. The agent picks a task and assigns itself. The task moves to `in_progress` with the profile name on it.

Execute. The agent does the work using any tool in its toolset, then moves the task to `review` or `done`, optionally adding notes about what it found.

This pattern scales to multiple profiles running concurrently. **The board is the shared state that coordinates them without requiring direct communication between agents.** That is the architectural point: no message bus, no RPC between profiles, just a database they all read.

Task lifecycle

Four mechanisms turn a list into a workflow.

MECHANISM	WHAT IT DOES	WHY IT MATTERS
Blocked tasks	A task lists its blocker task IDs and is not claimable until they resolve	Stops multiple agents spinning on work that cannot proceed
Deadlines	Optional; the task alerts as it approaches	Combined with priority, creates a natural triage order
Iteration limits	Caps how many tasks can sit in a given state at once	Three in review means review is the bottleneck, and the system stops pulling new work until it clears
Lifecycle notes	Appended on each state transition: what was done, found, and left undone	Accumulate into a complete audit trail; the reviewer reads them before starting

Iteration limits are the subtle one. A cap on `review` does not just throttle, it **surfaces where the workflow is slow**. A board that keeps hitting its review limit is telling you the reviewer is under-resourced, and that is information a chat transcript would never give you.

Multi-agent coordination

The board comes into its own when multiple profiles share it. Each profile is a separate agent with its own config, memory, skills and toolset, and the board is the bridge.

A typical setup has three:

- **The orchestrator** populates the board from an external source: a cron job reading an RSS feed, a gateway message creating a task, or a manual request.

- **The worker** polls for new tasks, claims them and moves them through the pipeline.
- **The reviewer** checks completed work and either approves it or sends it back for rework.

Each runs independently. The orchestrator does not wait for the worker; the worker does not wait for the reviewer. **The board is the synchronization point.** If the worker is offline, tasks pile up in `todo`, and when it returns it picks up where it left off.

The kanban tutorial in the Hermes docs walks through wiring three profiles with shared Raft state for distributed consistency: profiles on different machines, different networks, with no direct communication channel, coordinating through the board.

When kanban beats direct chat

Single-agent work is simpler. You ask, the agent does, you get the result. No board, no task states, no profile coordination. Kanban adds overhead, and every step of that overhead is friction direct chat does not have.

The board earns its overhead when:

CONDITION	WHY THE BOARD WINS
Work is long-running	A durable board survives agent restarts; direct chat loses context when the session ends
Work is multi-step	Explicit state transitions give each step a clear owner and completion signal
Work is multi-agent	Profile routing means the orchestrator does not need the worker's skills
Work is asynchronous	A task created at midnight is claimed by a worker at 8 AM. The board is always on

The board as system memory

A board with full task history is a record of what was done, when, by whom, and what was found. Every moved task, every lifecycle note, every state change persisted in SQLite. It answers questions direct chat cannot: how many tasks completed this week, which profiles are overloaded, where the pipeline bottlenecks are.

The board does not replace memory or skills, it completes the set.

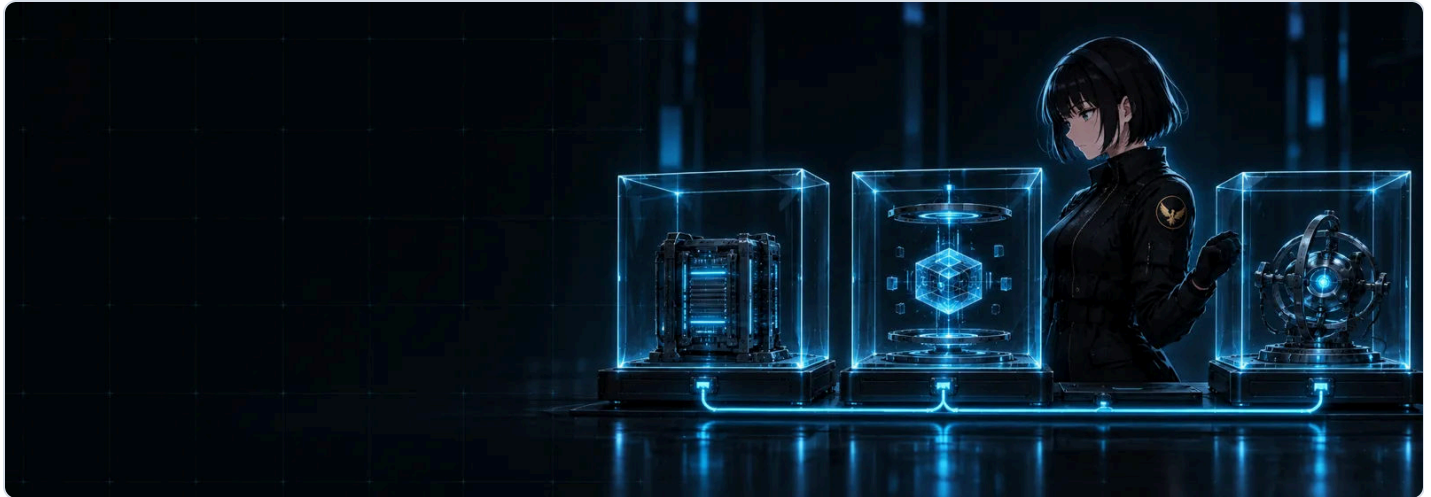
SYSTEM	HOLDS	TIME HORIZON
Memory	Durable facts	Always, in every prompt
Skills	Procedures	On demand, when matched
Kanban board	The state of in-flight work	Until the work is done

Together the three give the agent awareness of **what it knows, how to do things, and what it should be working on.**

OPERATOR DRILL · A MINIMUM VIABLE PIPELINE

Create two profiles, a worker and a reviewer. Put three real tasks on the board in `todo`. Have the worker claim and execute them, writing a lifecycle note on each transition. Then have the reviewer read only the notes, not the conversation, and decide whether each passes. If the reviewer can judge the work from notes alone, your lifecycle notes are good enough to coordinate agents. If it cannot, the notes are the thing to fix before you add a third profile.

The Admin Layer



One agent is a tool. Two agents is a system. Three or more is an operation.

As soon as you run multiple Hermes profiles, one for coding, one for research, one for the gateway, one as a kanban worker, you need a layer that manages them. **Profiles are the runtime boundary. The dashboard is the control surface. The API server is the integration point.**

Profiles are the runtime boundary

Each Hermes profile is a completely independent agent: its own `config.yaml`, its own `HERMES_HOME`, its own memory, sessions, skills, cron jobs, gateway state and credentials. `hermes -p coder chat` and `hermes -p researcher chat` start completely separate sessions. **They share nothing by default.**

```
profiles.sh
```

```
hermes profile create coder                # blank profile, fresh config
hermes profile create work --clone-from default # copies config, skills, SOUL
hermes profile create backup --clone-all      # full snapshot incl. memories,
                                              # sessions and cron jobs
hermes profile export coder                # shareable archive (no credentials,
                                              # no session history)
hermes profile install <archive>           # install it on another machine
```

Each profile automatically gets its own command alias. Create a profile called `coder` and `coder chat` launches Hermes in it. You never type `hermes -p coder` again.

Gateways are per-profile too. Each profile runs its own gateway as a separate process with its own bot token. Profile A can talk to Telegram while Profile B talks to Discord. If two profiles accidentally use the same bot token, the second gateway refuses to start with a clear error naming the conflict.

HOME_MODE IS THE ISOLATION SETTING PEOPLE MISS

By default **all profiles share your real home directory** for tool execution. Same git config, same npm state, same SSH keys. Setting `terminal.home_mode: profile` in a profile's config scopes tool execution to that profile's own home directory. Useful when you want a kanban worker to have its own git identity. The tradeoff is that tools like `git` and `gh` need re-authenticating per profile.

The dashboard is the control surface

The web dashboard runs as an optional web server giving a graphical view of your installation: current profile, active sessions, model configuration, tool settings, gateway status, skill library and cron jobs.

Its main value is the **profile switcher**. Switch between profiles, inspect each one's config, edit skills, adjust model settings and start chat sessions from one browser tab. You can see what the coder profile is working on while chatting from the researcher profile.

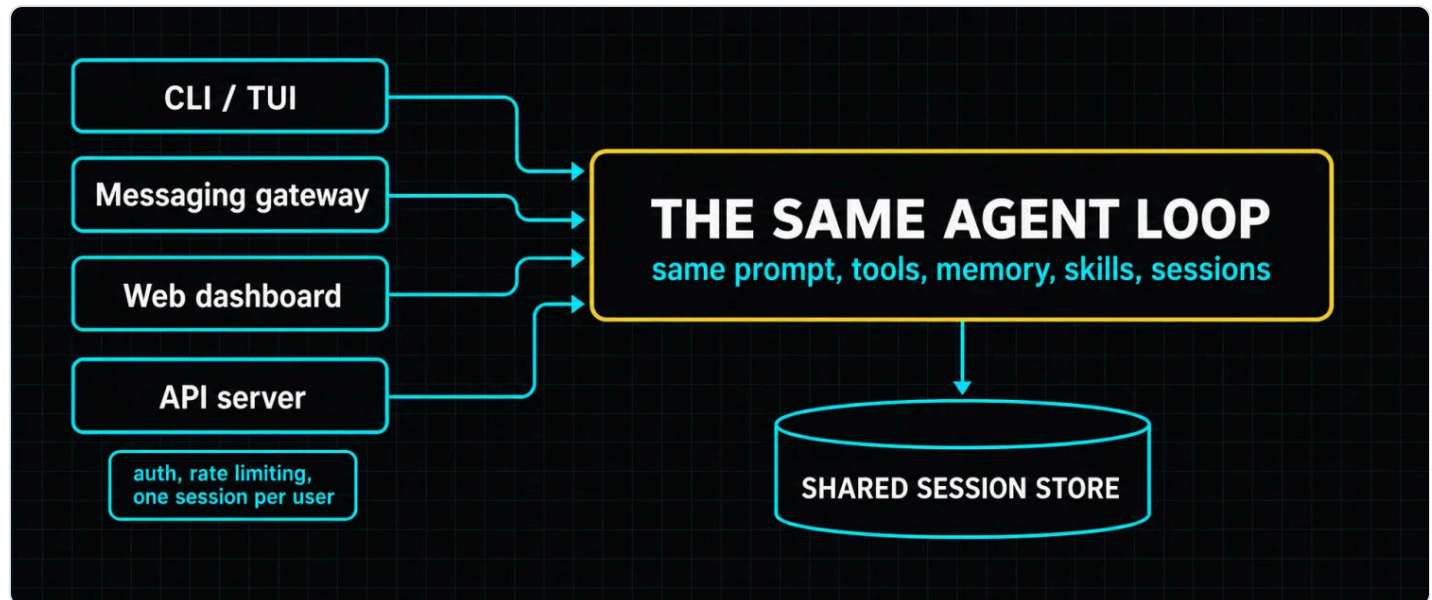
From that one tab you can:

- ✓ Switch between profiles and see each one's config, model, tools and gateway status
- ✓ Edit skills in the browser with syntax highlighting and live preview
- ✓ Manage cron jobs: create, edit, pause, resume, remove, without CLI commands
- ✓ Inspect session history across profiles and surfaces, all from the same session store
- ✓ Monitor active sessions, running tool calls, gateway status and recent completions

The dashboard is **optional**. Everything it does can be done through the CLI. For operators running multiple profiles with multiple gateways, the visual view saves time; for a single-profile setup it is a convenience you can skip.

The API server is the integration point

Hermes exposes an **OpenAI-compatible HTTP endpoint**. Any frontend that speaks the OpenAI format can drive it: Open WebUI, LobeChat, LibreChat and similar.



The only difference between surfaces is the transport. The API server routes requests through the active profile's agent loop with the same prompt assembly, tool dispatch and session persistence the CLI uses. That is what makes a custom frontend or a team-shared interface cheap to build: you are not reimplementing the agent, only changing how messages arrive.

What changes with multiple profiles

Single-profile Hermes is simple: one agent, one config, one set of skills, one gateway, everything in one place.

Multi-profile Hermes changes the operational model. The coding profile has terminal access and a capable model. The research profile has web tools and a cheap model for bulk work. The kanban worker has no gateway and a minimal toolset, running as a supervised background service.

Profiles share nothing by default, which is the safe default. They coordinate through the kanban board, or through the shared filesystem if you explicitly configure shared directories. **The board from Part 10 is the recommended path** because it does not require shared filesystem access.

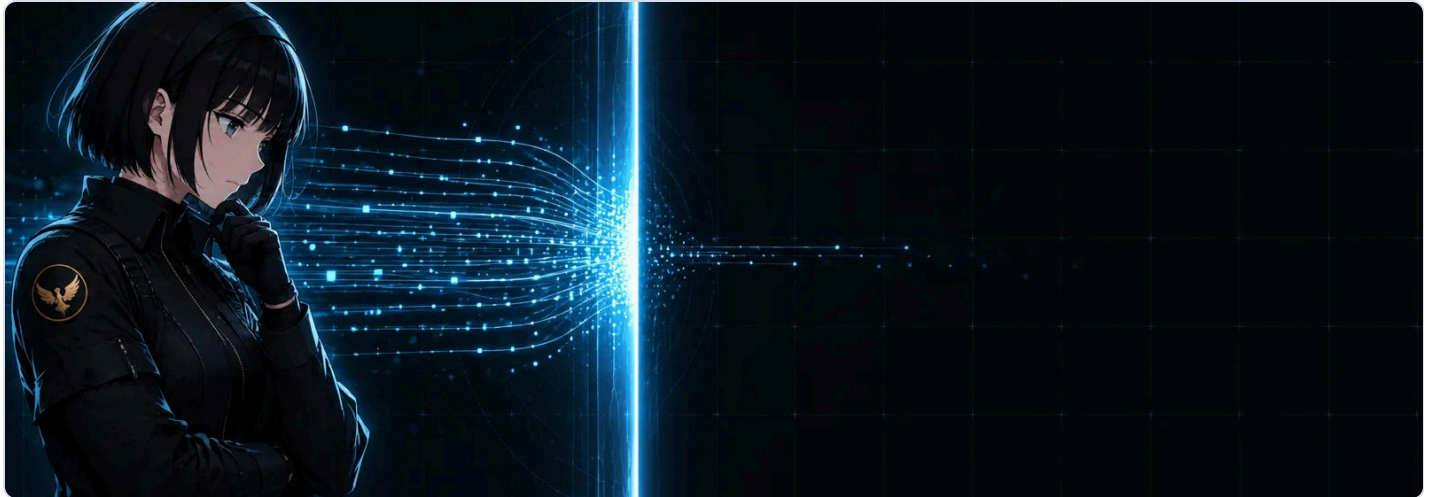
THE HONEST ADVICE ON WHEN TO ADD A PROFILE

More profiles means more config files, more gateways to manage, more things that can break. Start with one and add specialized ones only when you have a clear use case. A general-purpose profile with all tools enabled plus a research profile with a cheap model is usually enough for most setups. Profile sprawl is a real failure mode and it arrives quietly.

OPERATOR DRILL · PROVE ISOLATION IS REAL

Clone your default profile with `--clone-from`. In the clone, set `terminal.home_mode: profile`. Then run the same command in both, something that reveals identity, `git config user.email` works well. Different answers mean isolation is real. Identical answers mean you are running two agents that can reach into each other's world, which is fine if you chose it and dangerous if you assumed otherwise.

What Breaks, What to Skip, How to Stay Sane

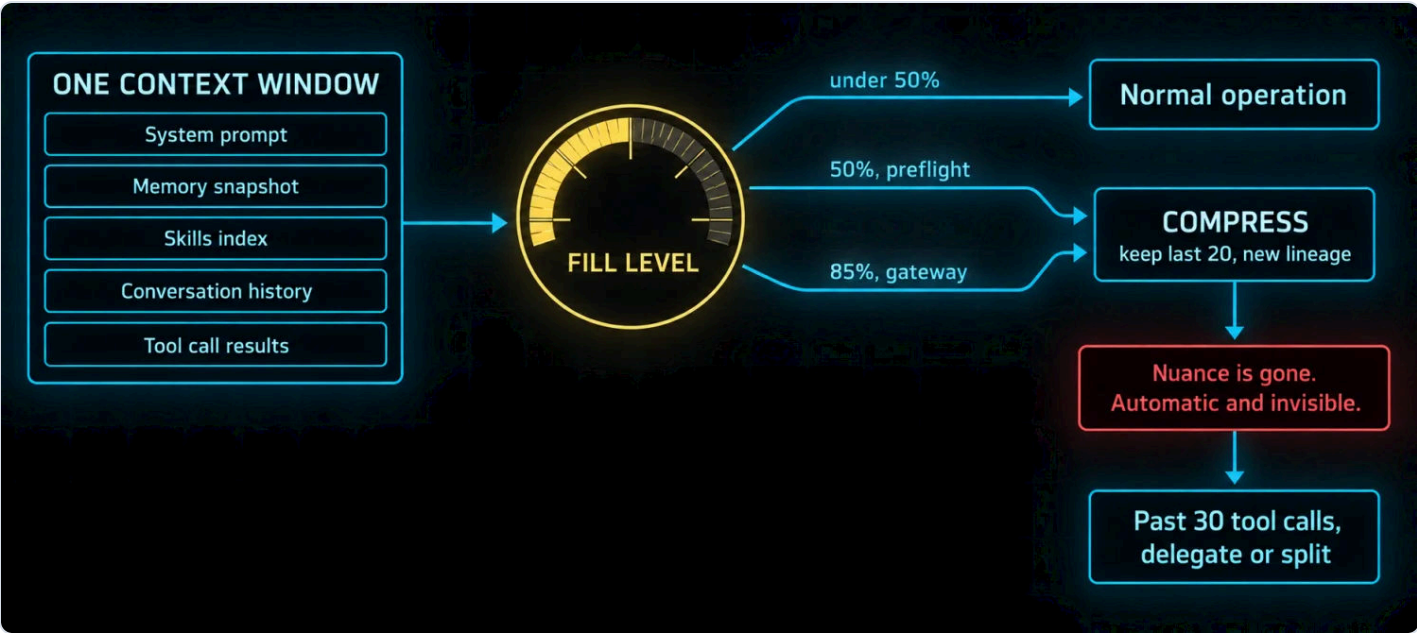


Eleven parts of features, tools and possibilities. Now the real talk.

Hermes is the most capable open-source agent the author has used. It is also a complex system with sharp edges. **Understanding what breaks, what to skip, and when to use something else is the difference between an agent that compounds and one that frustrates.**

Context windows are the first wall

Every model has a context window and Hermes runs inside it.



The system prompt, memory snapshot, skills index, conversation history and tool call results all compete for the same limited space.

THRESHOLD	WHAT FIRES	WHAT IT COSTS
50% of window	Preflight compression, before the API call	Middle turns summarized, last 20 preserved, new session lineage
85% of window	Gateway auto-compression	Same mechanism, later trigger
Every compression	Nuance	Compressed content is a summary, not the original. Automatic, usually invisible, and destructive

Long tool call chains accelerate the pressure. A single research task might produce five search results, three page extracts and a summary: nine call-and-result pairs in history. Run another batch and the history doubles. Preflight compression catches it, but the agent loses nuance with every pass.

The iteration budget is the safety valve. Default 90 turns, each tool call counting as one. **Subagents have their own budgets, so delegation is a genuine workaround for tight context** because a child's iterations do not count against the parent.

THE PRACTICAL RULE, STATED PLAINLY

If a task needs more than 30 tool calls, delegate it to a subagent or break it into multiple sessions. Do not let a single conversation chain grow until it compresses into illegibility. By the time you notice the agent has gone vague, the detail it lost is already gone.

Integration fragility is constant

Every tool depending on an external service is a point of failure. Web search needs an API key. The browser tool needs a cloud provider credit balance or a running local Chromium. Image generation needs the FAL client installed. TTS needs an ElevenLabs or similar key.

Credentials expire. Rate limits hit. Free tiers run out. The tool's `check_fn` reports unavailable and **the model silently loses access. No error. No alert.** The tool just does not appear in the schema, exactly as Part 5 described.

FALLBACKS COVER THE MODEL, NOT THE TOOLS

The fallback provider system handles model failures: on a 429 or 5xx, Hermes tries the next provider. But fallbacks only cover the model. If your web search API key expires, the search tool disappears regardless of which model you are running. There is no tool-level fallback, so there is no substitute for checking.

The practical rule: run `hermes tools` periodically to verify your surface is intact, and **set up a weekly cron job that tests the three tools from Part 2**, terminal, web and file, alerting you if any fail. A tool that disappears silently is worse than a tool that never worked, because you built on it.

Profile isolation has footguns

Profiles give you independent agents. They also share your actual system by default.

FOOTGUN	THE SYMPTOM	THE FIX
Shared home directory	A coding profile's git config is the research profile's git config; profiles can read each other's project files	<code>terminal.home_mode</code> : profile scopes execution to <code>{HERMES_HOME}/home</code> . Tradeoff: re-authenticate git and gh per profile
Gateway token conflict	The second gateway refuses to start	Create separate bots for separate profiles
Session persistence	Docker without the volume mount loses everything on restart; serverless loses active sessions until wake	Test it: start a chat, stop, restart, <code>hermes -c</code>

Profiles are config-isolated, not system-isolated. That distinction is the whole of this section, and it surprises people who assumed a profile was a sandbox.

The curator can bite

The curator prevents skill rot by archiving skills unused for 90 days, running automatically every 7 days. That is great for agent-created skills accumulated from a dozen workflows. It is annoying when it archives a skill **you** wrote and use once a quarter.

The fix is one command: `hermes curator pin <name>`. Pinned skills leave the curator's jurisdiction entirely. The agent can still patch them; the curator cannot archive them.

NOTHING IS EVER DELETED, SO CHECK THE ARCHIVE FIRST

Archived skills live in `~/.hermes/skills/.archive/` and come back with `hermes curator restore <name>`. If a skill disappears, it is almost certainly there. And pin anything you wrote yourself, as a habit, on the day you write it.

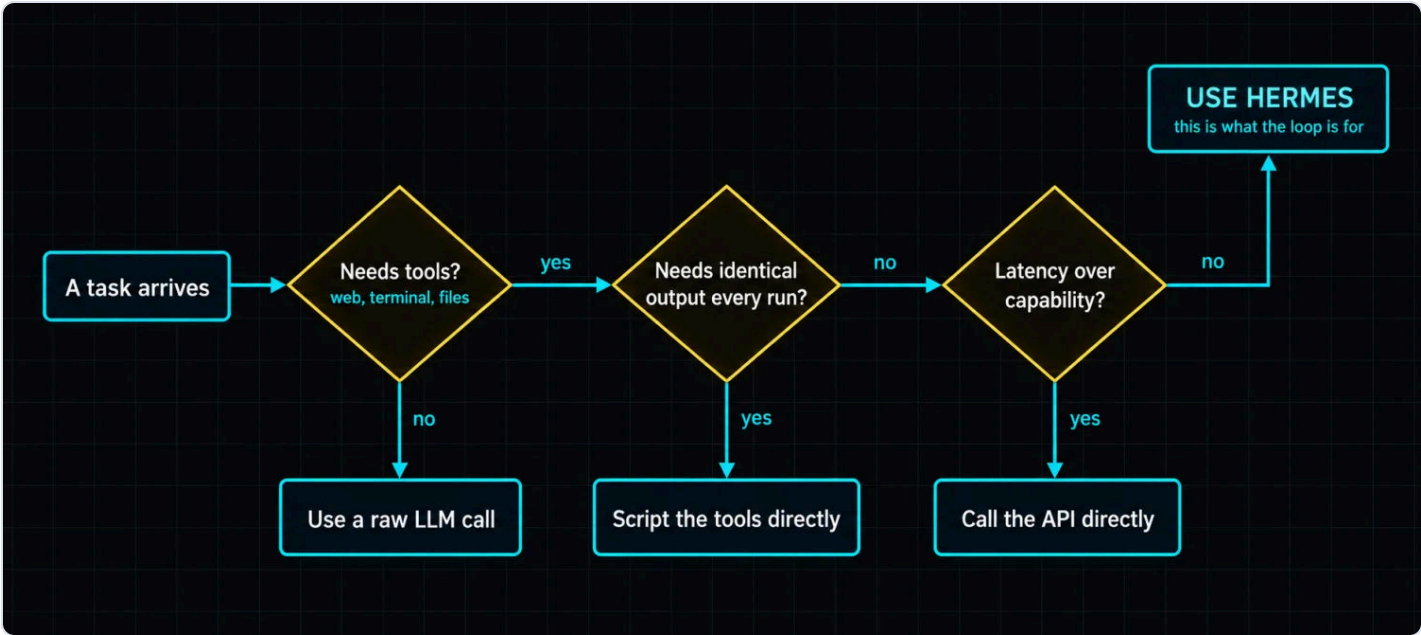
What to skip

Not every feature is worth your time on day one. Some are genuinely useful for advanced cases. Some are distractions until you have a specific need.

SKIP	UNTIL
MCP servers	You need a specific integration. Core tools already cover web, terminal, file, browser and media
Multi-provider routing	Your primary model fails regularly. One provider works fine for months
Batch processing	You are training models or doing research at scale
Custom plugin development	The built-in tools genuinely do not cover your case. A skill covers most procedural needs without code
ACP editor integration	You live in VS Code or Zed and want Hermes inside the editor. CLI and gateway cover the same ground

When not to use Hermes at all

Hermes is an agent with a tool surface. It is not the right tool for every task, and knowing the boundary is part of using it well.



DO NOT USE IT FOR	BECAUSE	USE INSTEAD
Single-shot generation	The agent loop adds overhead a single inference does not need	A raw LLM call

DO NOT USE IT FOR	BECAUSE	USE INSTEAD
Tasks that need no tools	"Translate this paragraph" needs no web, terminal or file access	A ChatGPT or Claude chat
Deterministic output	The loop is inherently non-deterministic: different tools, orders and approaches each run	Script the tools directly
Latency-critical work	Every tool call adds round-trip time on top of model reasoning	Call the API directly

What compounds, and what breaks it

The eleven parts before this one describe a system that gets better over time: the agent loop, session persistence, memory and skills, the tool surface, cron, gateways, delegation, browser automation, kanban, profiles.

None of it works if the foundation is wrong. Sessions that do not persist break the learning loop. Missing API keys break the tool surface. Gateway tokens in the wrong profile break message delivery. The curator archiving a skill you wrote breaks an automation you relied on.

The agent loop is the engine. The learning system is the fuel. The tools are the output. Profiles are the runtime. And the limits are what keep it real.

- ☐ Run hermes tools this week and read the whole list
- ☐ Pin every skill you wrote yourself
- ☐ Test session persistence: start, stop, restart, hermes -c
- ☐ Create the weekly three-tool canary cron job
- ☐ Check ~/.hermes/skills/.archive/ for anything you miss

OPERATOR DRILL · BUILD THE CANARY

Write one cron job, in no-agent mode where possible, that exercises terminal, web and file once a week and reports only on failure using the `[SILENT]` pattern from Part 6. This single job converts the entire "integration fragility" section from a thing you have to remember into a thing the system tells you. It is the highest-value twenty minutes in this document.

Complete Command Index

Every command the twelve parts name, collected. Commands are grouped by what you are trying to do rather than by subsystem, because that is how you will look for them.

[illegible]


```
/model                # switch model (rebuilds the prompt)
/agents               # TUI live tree of the delegation fan-out
/github-pr-workflow /test-driven-development <task>  # stack two skills
```

Every Path and What Lives There

The ~/.hermes tree, as the series describes it

~/.hermes/	the entire agent, one directory
config.yaml	per-profile configuration
state.db	SQLite: sessions, titles, FTS5 search, lineage
MEMORY.md	agent notes · ~2,200 char hard limit
USER.md	your profile · preferences and style
SOUL.md	agent identity (stable prompt tier)
skills/	
<category>/<name>/SKILL.md	one skill, frontmatter + four sections
.archive/	curator-archived skills (recoverable)
cron/	
jobs.json	the job store, atomically written
.tick.lock	prevents overlapping 60s ticks
output/<job_id>/<timestamp>.md	every run's output, kept for audit
kanban/	SQLite task board for multi-agent work
home/	per-profile home when home_mode: profile

(in Docker, all of the above is mounted at /opt/data)

project directory/	context tier, ONE of these, by priority
.hermes.md	1st, wins over the others
AGENTS.md	2nd
CLAUDE.md	3rd, only if neither above exists

Every Number in One Table

The series scatters its constants across twelve articles. Here they are together.

CONSTANT	VALUE	WHERE IT APPLIES
Registered tools	70+	Across ~28 toolsets, excluding your MCP servers
Providers supported	18+	With OAuth flows and credential pools
API execution modes	3	OpenAI chat completions, OpenAI Codex/Responses, native Anthropic Messages
Terminal backends	6	local, Docker, SSH, Singularity, Modal, Daytona
Browser backends	5	Browserbase, Browser Use, Firecrawl, local CDP, managed Chromium
Messaging platforms	20+	One gateway process serves all of them
Main iteration budget	90 turns	Each tool call is one turn
Subagent iteration budget	50 turns	Per child, independent of the parent
Default concurrent children	3	Per delegation batch
Default orchestration depth	1 (flat)	Depth 3 at width 3 means 27 leaf agents
Preflight compression	50% of context	Before the API call
Gateway auto-compression	85% of context	During a gateway session
Messages preserved on compression	Last 20	Middle turns are summarized
Memory budget	~1,300 tokens	MEMORY.md plus USER.md combined

CONSTANT	VALUE	WHERE IT APPLIES
MEMORY.md hard limit	2,200 characters	Returns an error with current entries, never a silent drop
Skill index cost	~3,000 tokens	For a library of dozens of skills
Curator tick	Every 7 days	After at least 2 hours idle
Skill goes stale	30 days unused	Deterministic phase
Skill archived	90 days unused	Recoverable, never deleted
Cron tick	Every 60 seconds	Gateway-hosted scheduler
Screenshot cost	~1,500 tokens	Flat rate on Anthropic regardless of base64 length
Screenshots kept in context	3 most recent	Older become placeholder text
Computer-use session cost	~30K tokens	Versus ~600K unoptimized
Removed child timeout	Formerly 300s	Now heartbeat staleness detection; hard timeout is opt-in

Failure Triage

The symptom you will actually observe, mapped to the cause and the check. Compiled from the failure modes named across Parts 2, 5, 6, 11 and 12.

SYMPTOM	LIKELY CAUSE	THE CHECK
Agent forgets yesterday's conversation	Docker running without the <code>~/hermes</code> volume mount	Start a chat, stop, restart, run <code>hermes -c</code>
A tool the docs describe simply is not there	<code>check_fn</code> is failing: expired key, exhausted credit, unreachable backend	<code>hermes tools</code> and read the whole list
"all" toolset is on but kanban is missing	Specialist toolsets are opt-in alongside all	<code>hermes tools</code> , then add <code>kanban</code> explicitly
Your CLAUDE.md is being ignored	A <code>.hermes.md</code> or <code>AGENTS.md</code> in the same directory wins	Check which of the three exist
A cron job behaves as if its skill does not exist	The skill is not in the DEFAULT profile, which the gateway uses	Copy it to the default profile's skills directory
A cron job says it has no idea what you mean	The prompt is not self-contained; cron sessions have zero history	Rewrite the prompt with every detail inline
A subagent asks about context it should have	The goal field assumed parent history the child never had	Put file paths, errors and conventions in goal and context
The second gateway refuses to start	Two profiles sharing one bot token	Create a separate bot per profile
The research profile can read your coding project	Default <code>home_mode</code> shares your real home directory	Set <code>terminal.home_mode</code> : profile
A skill you wrote has vanished	Curator archived it after 90 days unused	Check <code>~/hermes/skills/.archive/</code> , restore, then PIN it

SYMPTOM	LIKELY CAUSE	THE CHECK
The agent has gone vague on a long task	Compression has fired and summarized the middle	Delegate or split the session past ~30 tool calls
Your local dev server appeared in a cloud provider log	Hybrid routing is not configured	Verify private addresses route to the local Chromium sidecar
One platform went quiet, the rest work	Circuit breaker tripped on that adapter	hermes gateway resume <platform>

A 30/60/90 Adoption Path

An addition, not from the source. The series is ordered by subsystem; this is the same material ordered by what to do first.

WINDOW	DO THIS	PARTS
Days 1-7	Pick a deployment home. Run the three-tool smoke test. Prove session persistence by restarting. Do nothing else	1, 2
Days 8-30	Let memory accumulate. Write two skills for workflows you have already explained twice. Audit your tool surface once	3, 4, 5
Days 31-60	Add cron: one no-agent watchdog, then one skill-backed digest. Wire one messaging gateway. Build the three-tool canary	6, 7, 12
Days 61-90	Use delegation for your first real parallel research task. Add browser work if you need it. Only now consider a second profile	8, 9, 11
Past 90	Add a kanban board only when you genuinely have multi-profile, long-running work to coordinate	10

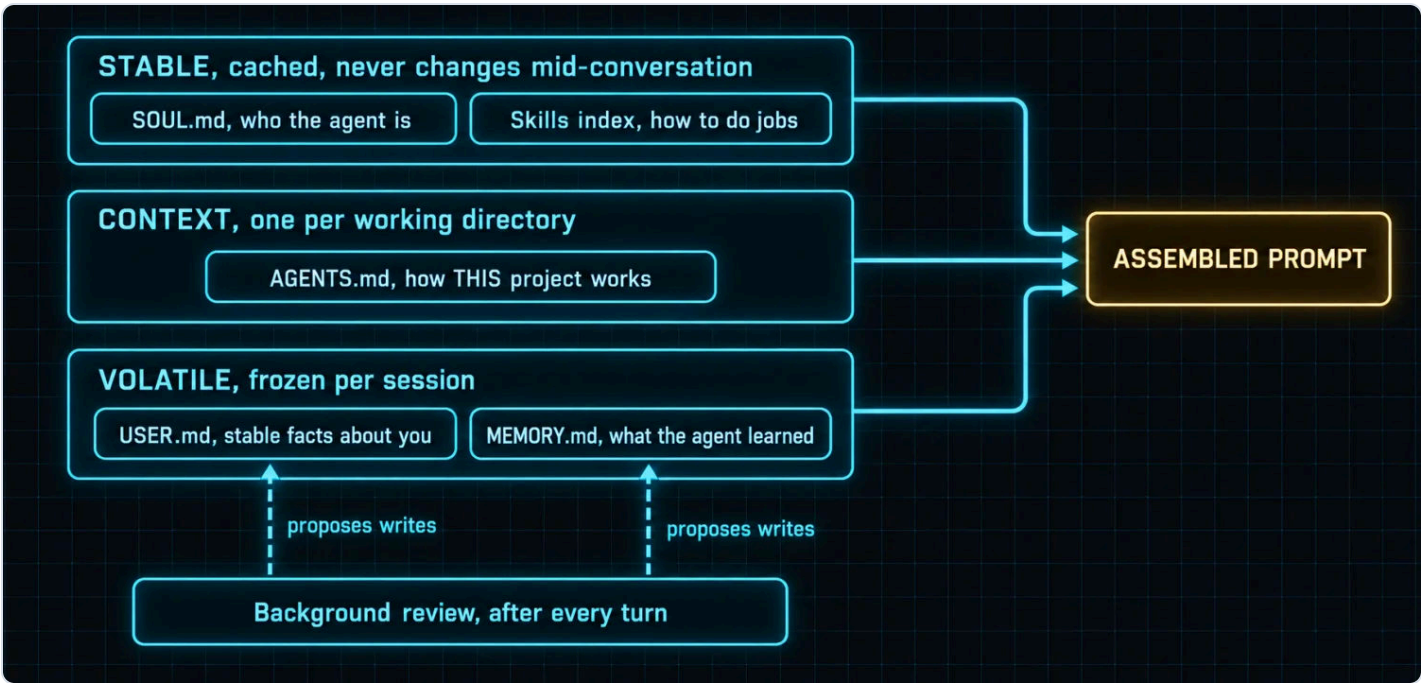
The ordering principle: **every stage must be provably working before the next one is allowed to depend on it.** That is the actual thesis of Part 2 restated as a calendar.

Personality and Profile, SOUL.md and USER.md

Parts 1 and 3 explain where these files sit in the prompt. Neither shows what to put in them, which is the gap most people fall into: they write a long adjective-laden character sketch, watch the agent ignore it, and conclude that identity files do not work.

They do work. They are just not the place for adjectives.

Three files, three different jobs



FILE	ANSWERS	WRITTEN BY	BUDGET
SOUL.md	Who are you, how do you behave, what do you never do	You, deliberately	No hard cap, but it is in EVERY prompt
USER.md	Who am I, what do I prefer, what should you never ask twice	You, plus the background review	Shares the ~1,300 token memory budget
MEMORY.md	What has the agent learned about my environment and projects	The agent, via the background review	2,200 characters, hard

FILE	ANSWERS	WRITTEN BY	BUDGET
AGENTS.md or .hermes.md	How does THIS project work	You, per project	Per working directory, one file wins

THE SINGLE MOST COMMON MISTAKE

Putting project facts in SOUL.md. Identity is stable and global; project facts belong in that project's context file, where they load only when you are working there. A SOUL.md that names your current client is a SOUL.md that will be wrong in three months and will still be costing tokens in every unrelated conversation.

What actually belongs in SOUL.md

Write behavior, not personality. The test for every line: **could an observer tell whether the agent followed it?** "Be insightful" fails that test. "When I ask for a recommendation, give one option and the reason, not a list of five" passes it.

A SOUL.md that works has roughly these parts, and stays short enough to read in one screen.

Identity

You are my working agent. You run tasks end to end and report what actually happened, not what was attempted.

How you behave

- Lead with the answer, then the reasoning. Never the reverse.
- When you recommend something, recommend ONE option and say why. If the choice is genuinely close, say that in a sentence and still pick one.
- Report failures as plainly as successes. A task that half worked is a task that failed; say which half.
- If a claim can be checked with a tool, check it before stating it.
- Ask a question only when the answer changes what you would do. Otherwise state your assumption and continue.

What you never do

- Never send, publish, delete, or spend money without my explicit go in the conversation where it happens.
- Never present a guess as a finding. Label uncertainty out loud.
- Never silently drop part of a task. If you skipped something, say so.

Escalation

If you hit something ambiguous mid-task, finish everything that does not depend on the ambiguity, then ask one specific question about the part that does.

WHY THIS SHAPE SURVIVES CONTACT WITH THE MODEL

Every line is a rule with an observable outcome, so a future turn can be judged against it. The "what you never do" block is the load-bearing half, because prohibitions are what stop an agent taking an irreversible action at 2 a.m. on a cron schedule. Note it also names the approval boundary, which is exactly what the two worked prompts in Appendix H rely on.

What actually belongs in USER.md

USER.md is the answer to "what should you never make me say twice". It is not a biography. It shares a roughly 1,300-token budget with MEMORY.md, so every line is competing with something the agent learned on its own.

```
~/hermes/USER.md
```

```
# Who I am
```

```
Time zone: America/New_York. Working hours 07:00 to 19:00.
```

```
Primary machine: macOS. Shell: zsh.
```

```
# How to talk to me
```

- I dictate, so expect typos and run-on sentences. Read for intent and do not comment on the typos.
- Short answers. If it fits in three sentences, use three sentences.
- I want the recommendation, not the survey.

```
# Standing preferences
```

- Package manager: bun, never npm.
- Language: TypeScript over Python unless I say otherwise.
- US spelling in everything you write for me.
- Never commit secrets. Sweep before any first push.

```
# Things I do not want asked again
```

- My email address is on file; do not ask for it.
- Default repo visibility is private. Going public is always an explicit call.

The interview beats the blank page

Writing these cold produces adjectives. The reliable method is to let the agent draft them from evidence and then edit what it got wrong. This is a prompt you can paste as-is.

Interview me so we can write SOUL.md and USER.md properly, then write them.

Do not ask me fifty questions and do not ask for anything you can already infer from our history in this session.

Run it in short rounds, one subject per round, and start each round by telling me what you already believe so I only have to correct you:

1. Outcomes. What am I actually trying to get out of working with you?
2. Working style. How do I want answers shaped, and what annoys me?
3. Initiative. What should you just do, what should you propose first, and what must always stop for my explicit go?
4. Boundaries. What should you never do, never store, and never say?
5. Standing facts. What should you never make me repeat?

Ask me for concrete examples rather than adjectives: one answer of yours I liked, one I did not, one decision you should have made alone, and one you should have checked with me first.

Then write two files and show me both before saving anything:

- SOUL.md, containing only behavior rules an observer could check, plus an explicit list of what you never do.
- USER.md, containing only stable facts and preferences that would otherwise make me repeat myself.

Keep project-specific facts OUT of both. Those belong in that project's context file. Tell me which things you deliberately left out and where they should live instead.

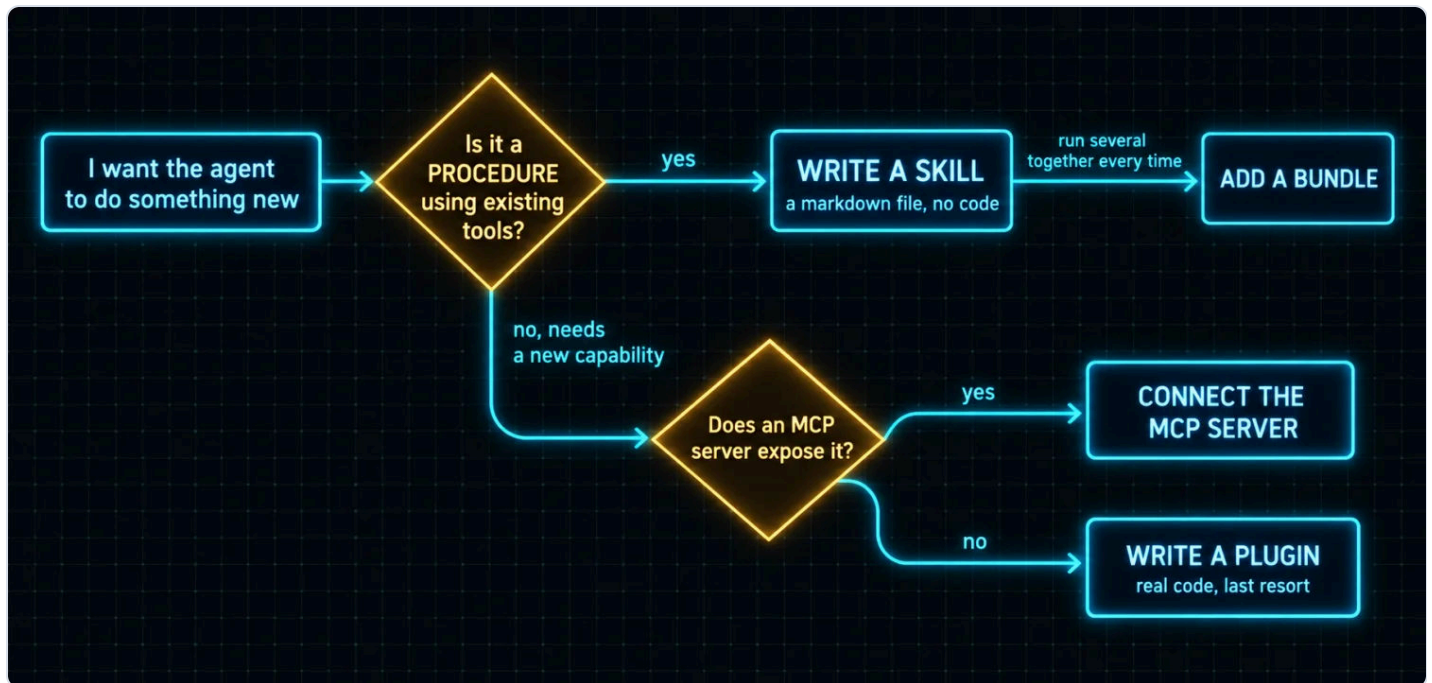
OPERATOR DRILL · TEST THE IDENTITY, DO NOT ADMIRE IT

After writing SOUL.md, give the agent a task that should trip one of its "never" rules, for example asking it to push something without saying go. If it stops and asks, the file is doing work. If it complies, the rule is decoration and needs rewriting as a concrete prohibition. An identity file you have never tested is a file you are hoping about.

Skills and Plugins by Example

Part 4 covers the anatomy of a skill and why progressive disclosure lets a library scale. This appendix is the practical companion: complete working files you can copy, the difference between the three things people confuse, and the rule for when a plugin is actually warranted.

Skill, bundle, or plugin



The ordering is not arbitrary. A skill costs one markdown file and one index line. A plugin costs code you now maintain against a moving project. Part 12's advice is explicit: skip custom plugin development until the built-in tools genuinely do not cover the case.

A complete skill, with the parts that usually get skipped

The two sections people omit are Pitfalls and Verification, which are the two that decide whether the agent can run the procedure without you watching.

name: deploy-web

description: Deploy the web app to staging or production, verify it is actually serving, and roll back if it is not

version: 1.0.0

When to Use

When I ask you to deploy, ship, release, or push the web app to an environment. Not for library releases and not for database migrations.

Procedure

1. Confirm which environment. If I did not say, ask. Never assume production.
2. Run the test suite. If anything fails, stop and report; do not deploy.
3. Record the currently deployed version so a rollback target exists.
4. Build. If the build emits warnings about missing env vars, stop and list them.
5. Deploy to the named environment.
6. Wait 15 seconds, then run the Verification section below.
7. If verification fails, roll back to the version recorded in step 3, then report what failed. Do not retry the deploy automatically.

Pitfalls

- Staging uses port 2222 for SSH, not 22.
- The deploy API returns 202 before the release is live. A 202 is not success.
- A cached CDN response can look healthy while the origin is broken. Always verify with a cache-busting query string.
- Never deploy to production on a Friday after 15:00 without asking me twice.

Verification

- The health endpoint returns HTTP 200 with a cache-busting parameter.
- The version string served matches the version just deployed.
- The error log has no new entries in the 60 seconds after deploy.
- Report all three results explicitly. Two out of three is a failed deploy.

THE DESCRIPTION IS THE WHOLE TRIGGER

The agent scans descriptions at session start and loads the body only on a match.

"Deployment helper" will never fire. The description above names the verbs a person would actually use, which is what makes it findable. If a skill never seems to load, rewrite the description as the request it should answer, not as a summary of the file.

A bundle is an alias, not a skill

Bundles exist for combinations you run constantly. They do not replace the individual skills and they carry no procedure of their own.

```
~/hermes/skills/bundles/ship-it.yaml
```

```
name: ship-it
description: The full release path, review through deploy
skills:
  - code-review
  - run-tests
  - deploy-web
```

Running `/ship-it fix the login redirect` loads all three skill bodies and the agent follows all three sets of instructions against the one task.

Installing from the hub, safely

```
hub.sh
```

```
hermes skills browse           # what exists
hermes skills search postgres # find by topic
hermes skills inspect pg-backup # READ IT before you install it
hermes skills install pg-backup # installs, after the security scan
hermes skills tap add myorg/skills # a private team repo of SKILL.md files
```

READ BEFORE YOU INSTALL

A skill is instructions a model will follow with your whole tool surface attached. The hub runs a security scanner for exfiltration, prompt injection and destructive commands, and that is a real control, but `inspect` costs thirty seconds and shows you exactly what you are granting. Treat an unread skill the way you would treat a shell script from a stranger.

When a plugin is genuinely the answer

A plugin adds a tool to the registry. Reach for one only when all three of these are true:

- ☐ The capability does not exist in the 70+ built-in tools

- ☐ No MCP server already exposes it
- ☐ The agent needs to call it as a TOOL mid-reasoning, not follow it as a procedure

Plugins participate in the same registry pattern as built-in tools, which means they self-register and carry a `check_fn` that gates availability. That gate is the part worth writing carefully: a plugin whose `check_fn` always returns true will appear in the schema even when its dependency is missing, and the model will call it and fail. Make the check test the real dependency, not a config flag.

THE CHEAP PATH MOST PEOPLE MISS

Before writing a plugin, try `execute_code`. The agent can write a Python script that does the job and run it, which covers a large share of what people reach for plugins to do, with no code to maintain and no registry surface to keep working across upgrades.

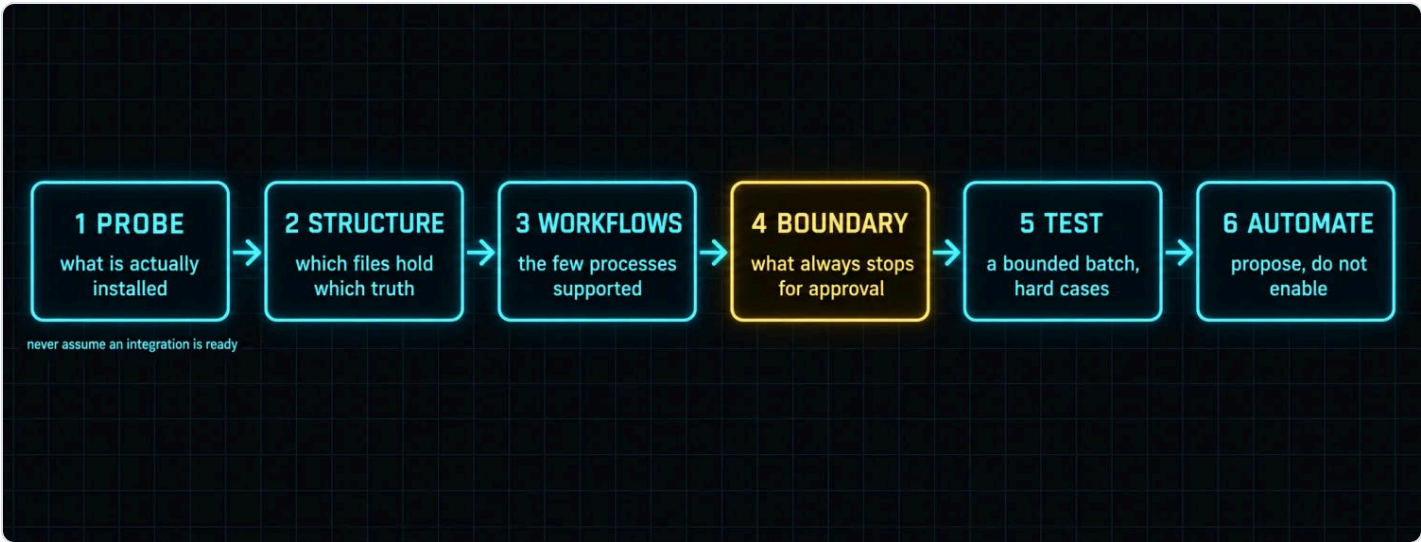
OPERATOR DRILL · MAKE ONE SKILL YOU ALREADY EXPLAINED TWICE

Find a workflow you have walked the agent through more than once. Write it as a SKILL.md with all four sections. Then open a brand new session and trigger it with a natural request that matches your "When to Use" wording, without naming the skill. If it loads, your description is right. If it does not, the description is a summary rather than a trigger, and that single fix is the difference between a library that compounds and a folder nobody reads.

Worked Build Prompts

Everything up to here describes what Hermes can do. This appendix is about how to ask for it. Both prompts below are real and both produce working systems, and the reason they work is a pattern you can reuse for anything.

The pattern underneath both



Six moves, and the order matters more than the wording. Most failed agent builds skip move 1 and move 5: they assume an integration is ready, and they automate before testing on real messy input.

MOVE	WHY IT IS IN THE PROMPT	WHAT GOES WRONG WITHOUT IT
Probe the environment	check_fn gating (Part 5) means capability is dynamic, not assumed	The agent designs around a tool that is not authenticated, then fails silently
Name the structure	Durable state must outlive a session (Part 2)	Follow-ups live in conversation memory and vanish on restart
Name the workflows	Bounded scope is what makes a skill writable (Part 4)	An unbounded assistant that does everything badly
Set the approval boundary	Cron and gateways run this unattended (Parts 6 and 7)	An agent that sends or deletes at 3 a.m.

MOVE	WHY IT IS IN THE PROMPT	WHAT GOES WRONG WITHOUT IT
Test on a bounded batch	Compression and misrouting only show on real input (Part 12)	Confident wrong classifications at scale
Automate last	Schedules are cheap to add and expensive to un-ring	A scheduled job nobody validated

Example one, an email inbox manager

prompt-inbox-manager.md

Build me a practical Hermes Email Inbox Manager in this working directory.

Start by checking which supported email connector is already available and authenticated. If none is ready, tell me the simplest supported setup for my email provider and ask only for what you actually need.

Create a minimal project structure for the Inbox Manager. Use AGENTS.md for stable project behavior, a separate rules file for my inbox classifications and approval boundaries, and durable follow-up state so waiting items do not depend on session memory.

The system should support four core workflows:

1. New Mail Triage
2. Reply Draft Queue
3. Follow-Up Watch
4. Daily Inbox Digest

Default to read + classify + draft. Do not send, delete, archive, or make other sensitive mailbox changes without my explicit approval unless I later define a specific safe policy.

Before automating anything, test the workflow on a small bounded batch of messages. Read full relevant threads, explain the classifications, prepare drafts for review, record follow-ups, and show any coverage gaps or failures.

Once the manual flow works, propose the smallest useful automation plan for a daily digest and follow-up checks. Do not enable schedules until I approve them.

Keep the build simple, inspectable, and easy to change later.

WHAT MAKES THIS ONE WORK

"Default to read + classify + draft" is the whole safety model in six words, and it maps onto a real Hermes boundary: drafting touches nothing irreversible, sending does. "Durable follow-up state so waiting items do not depend on session memory" is Part 2's lesson stated as a requirement rather than discovered after a restart loses a week of follow-ups. And the last line, keep it inspectable, is what stops the agent building a clever opaque system you cannot correct.

Example two, a contact and relationship manager

prompt-relationship-manager.md

Build me a Contact & Relationship Manager inside this Hermes workspace under projects.

First inspect the current environment and available tools/Skills. Do not assume Gmail, Calendar, Google Contacts, Cron, or any other integration is configured. Tell me what is available and ask only for information or access that is genuinely missing.

Use a transparent file-backed system as the default source of truth. Create the smallest useful structure with:

- AGENTS.md for the operating role,
- RELATIONSHIP_RULES.md for privacy, capture, identity, follow-up, and approval rules,
- CONTACT_INDEX.md for canonical contact lookup,
- FOLLOW_UPS.md for the active follow-up queue,
- a contacts/ folder with one Markdown record per maintained relationship.

The system should support four core processes:

1. interaction capture,
2. contact context updates,
3. follow-up reminders,
4. pre-conversation briefs.

Rules:

- match people by stable identifiers when possible and never merge on name alone,
- capture concise useful context instead of dumping full conversations,
- date source-grounded updates,
- keep facts separate from inference,
- do not infer sensitive personal traits,
- preserve interaction history,
- only create follow-up dates or cadences when there is a reason,
- keep outbound messages, calendar changes, and other consequential external actions under my approval,
- treat Google Contacts as an optional identity source, not as the relationship database unless the current environment proves richer write support,
- do not use Hermes built-in MEMORY.md or USER.md as the contact database.

If Google Workspace is available, verify the current authentication and scopes and use Gmail, Calendar, and Contacts only for the parts they currently

support, or a better option you already have.

Before automating anything, test the build with representative cases including a duplicate-name case, a resolved follow-up, and an upcoming conversation. Show me the results and fix any bad routing or overcapture.

Once the manual system works, propose a simple recurring relationship review that surfaces due follow-ups and useful pre-conversation briefs without sending anything automatically. Do not schedule it until I approve the exact behavior.

THE LINE THAT MATTERS MOST IN EXAMPLE TWO

"Do not use Hermes built-in MEMORY.md or USER.md as the contact database." Without it, an agent will happily start writing people into memory, and memory has a 2,200 character hard cap it will then silently fight against by consolidating away the very details you wanted kept. Appendix F covers what those two files are actually for. Domain data belongs in domain files.

WHY THE TEST CASES ARE NAMED EXPLICITLY

A duplicate name proves the identity rule. A resolved follow-up proves items leave the queue rather than accumulating forever. An upcoming conversation proves the brief actually assembles from stored context. Naming three adversarial cases is worth more than asking for "thorough testing", because the agent will otherwise test the happy path and report success.

A reusable template

Strip either example down and this is what remains. Fill the brackets.

Build me a [SYSTEM NAME] in [WHERE IT LIVES].

First inspect the current environment and available tools and skills. Do not assume [INTEGRATIONS] are configured. Tell me what is available and ask only for what is genuinely missing.

Use a transparent file-backed system as the source of truth. Create the smallest useful structure with:

- AGENTS.md for the operating role,
- [RULES FILE] for the policies and approval boundaries,
- [STATE FILES] for anything that must outlive a session.

The system should support these core processes:

1. [PROCESS]
2. [PROCESS]
3. [PROCESS]

Rules:

- [the domain rules that keep the data trustworthy]
- keep [the irreversible actions] under my approval.

Before automating anything, test with representative cases including [HARD CASE], [HARD CASE], and [HARD CASE]. Show me the results and fix any bad routing before we continue.

Once the manual system works, propose the smallest useful recurring job. Do not schedule it until I approve the exact behavior.

Keep the build simple, inspectable, and easy to change later.

OPERATOR DRILL · BUILD ONE OF THESE FOR REAL

Pick the one closer to a problem you actually have and run it as written. Watch specifically for what the agent does at move 1: if it starts building without telling you what is and is not authenticated, stop it and say so. That single habit, refusing to assume the environment, is the difference between a system that works on your machine and a system that works on the machine it was imagined on.

Sources

All twelve parts by Tony Simons (@tonysimons_), published on X between July 5 and July 20, 2026. Retrieved and compiled September 16, 2026.

PART	TITLE	PUBLISHED	LINK
Index	Hermes Agent Masterclass: The Whole Damn Thing	Jul 20, 2026	x.com/tonysimons_/status/207906642221117218
1	How Hermes Agent Actually Processes Work	Jul 5, 2026	x.com/tonysimons_/status/2073880068657471523
2	The Choices That Compound	Jul 7, 2026	x.com/tonysimons_/status/2074253518224109718
3	The Learning System	Jul 8, 2026	x.com/tonysimons_/status/2074634038007001223
4	Skills as Executable SOPs	Jul 9, 2026	x.com/tonysimons_/status/2075010900247843018
5	Tools and Toolsets	Jul 10, 2026	x.com/tonysimons_/status/2075526192476631476
6	Cron Makes Hermes Infrastructure	Jul 11, 2026	x.com/tonysimons_/status/2076011080229552207
7	Messaging Gateways Make Hermes Ambient	Jul 14, 2026	x.com/tonysimons_/status/2076838692828684573
8	Delegation and Subagents	Jul 15, 2026	x.com/tonysimons_/status/2077219246178718068
9	Browser and Computer Use	Jul 17, 2026	x.com/tonysimons_/status/2077923978249724073
10	Kanban as a Coordination Model	Jul 18, 2026	x.com/tonysimons_/status/2078279982845980737

PART	TITLE	PUBLISHED	LINK
11	The Admin Layer	Jul 19, 2026	x.com/tonysimons_/status/2078641430562492927
12	What to Skip, What Breaks, and How to Stay Sane	Jul 20, 2026	x.com/tonysimons_/status/2078972949483454619

The author also references two companion pieces worth reading alongside the series: a guide to your first two weeks with Hermes Agent, and an earlier deep dive on kanban from May 2026. Official installation documentation lives at hermes-agent.nousresearch.com/docs/getting-started/installation.

WHAT THIS DOCUMENT ADDED

The prose substance, and every mechanism, threshold and command above, come from the source articles. Added while compiling: twenty-three diagrams, thirteen illustrations, the consolidated constant and triage tables, the vocabulary table, the operator drills, the cross-references between parts, the 30/60/90 path in Appendix E, and the configuration and prompt guidance in Appendices F, G and H. Nothing was invented about how Hermes behaves; where a number appears it came from the source.